CENTRAL PARK HOUSING SOCIETY MANAGEMENT SYSTEM

A Comprehensive Web-Based Residential Administration, Maintenance & Gate Security Portal

A Project Report

Submitted in partial fulfillment of the requirements for the award of the Degree of

BACHELOR OF SCIENCE (COMPUTER SCIENCE)

By

Neha Shridhar Kadam

Seat No:

Under the esteemed guidance of

Mrs. Mona Khobare

ASST. PROFESSOR



DEPARTMENT OF COMPUTER SCIENCE

MODEL COLLEGE OF SCIENCE AND COMMERCE

(Affiliated to University of Mumbai)

KALYAN, 421306

MAHARASHTRA

2026

PROFORMA FOR THE APPROVAL OF PROJECT PROPOSAL

PNR No.: _____ Roll No.: _____

1. Name of the Student: Neha Shridhar Kadam
2. Title of the Project: Central Park Housing Society Management System
3. Name of the Guide: Mrs. Mona Khobare
4. Teaching experience of the Guide: _____
5. Is this your first submission? Yes / No

Signature of the Student:
_____

Date: _____

Signature of the Guide:
_____

Date: _____

Signature of the Coordinator:
_____

Date: _____

The East Kalyan Welfare Society's

MODEL COLLEGE OF SCIENCE & COMMERCE, KALYAN(E)

Accredited 'B' Grade by NAAC
(Affiliated to University of Mumbai)

This is to certify that Mr./Miss. Neha Shridhar Kadam Seat No. _____ PRN No. _____ of T.Y.B.Sc (Computer Science) during Semester – V has completed the prescribed practical's of the course Central Park Housing Society Management System under our supervision in Model College of Science & Commerce, Kalyan(E), affiliated to Mumbai University during the Academic Year 2025–2026.

_____ _____

TEACHER IN-CHARGE EXTERNAL EXAMINER

_____ _____

HEAD, PRINCIPAL
DEPARTMENT OF COMPUTER
SCIENCE

T.Y.B.Sc.CS, SEM-V

DECLARATION

I hereby declare that the project “Central Park Housing Society Management System” carried out at The East Kalyan Welfare Society’s Model College of Science & Commerce, Kalyan(E), has not been duplicated or submitted to any other university for the award of any degree. This project is submitted in partial fulfilment of the requirements for the award of the degree of BACHELOR OF SCIENCE (COMPUTER SCIENCE) as the Semester V project.

I further declare that the work presented in this report is my own work carried out under the guidance of the project guide.

Date: _____

Place: _____

Signature of Student: _____

(Neha Shridhar Kadam)

Role and Responsibility Form

This project was undertaken individually. The role and responsibility of the student is summarised below.

Name	Role	Responsibility
Neha Shridhar Kadam	Sole Developer	Requirement analysis, relational database design, front-end and backend development, testing, and documentation of all modules of the Central Park Housing Society Management System.

Abstract

Central Park Housing Society Management System is a comprehensive web-based platform engineered to modernise, automate, and centralise residential housing society administration, resident member services, maintenance accounting, and checkpoint gate security operations. Developed using Python Flask, SQLite/MySQL relational database architecture, and a responsive Bootstrap 5 interface, the system implements a strict role-based access control paradigm supporting three discrete operational roles: Administrators, Residents, and Security Personnel. The Administrator manages apartment flats, residents, vehicle parking slots, generates periodic maintenance bills, reviews service complaints, broadcasts digital notices, and exports operational CSV reports. Resident members can securely view apartment occupancy details, check pending maintenance dues, make instant payments with automated receipt generation, lodge maintenance grievances, and inspect society circulars. Security guards operate an expedited gate console for logging visitor entry and executing one-click visitor checkout. The entire platform enforces secure password hashing, prepared SQL statements, session verification middleware, and CSRF protection. This blackbook report details the complete engineering lifecycle from requirements analysis and architectural modeling to empirical test verification and deployment.

Acknowledgement

I would like to express my sincere gratitude to my project guide, **Mrs. Mona Khobare**, for her constant guidance, valuable suggestions and encouragement throughout the development of this project. Her feedback was instrumental in shaping both the technical implementation and the presentation of this report.

I am also thankful to the Department of Computer Science and the faculty of Model College of Science and Commerce for providing the resources and environment necessary to complete this project.

Finally, I would like to thank my family and friends for their continuous support and encouragement throughout my academic journey.

Neha Shridhar Kadam

Table of Contents

Sr. No.	Index	Pg. No.	Sign
1	Original Copy of the Approved Proforma of the Project Proposal	i	
2	Certificate of Authenticated Work / Signature Page	ii	
3	Declaration	iii	
4	Role and Responsibility Form	iv	
5	**Abstract**	v	
6	**Acknowledgement**	vi	
7	**List of Figures**	ix	
8	**Chapter 1: Introduction**	1	
9	1.1 Background	1	
10	1.2 Objectives	1	
11	1.3 Purpose, Scope and Applicability	1	
12	1.3.1 Purpose	1	
13	1.3.2 Scope	1	
14	1.3.3 Applicability	1	
15	1.4 Achievements	2	
16	1.5 Organisation of Report	2	
17	**Chapter 2: Survey of Technologies**	3	
18	Front End	3	
19	Back End	3	
20	Database	3	
21	Development Environment	3	
22	**Chapter 3: Requirements and Analysis**	4	
23	3.1 Problem Definition	4	
24	3.2 Requirements Specification	4	
25	3.3 Planning and Scheduling	5	
26	3.4 Software and Hardware Requirements	8	
27	3.5 Preliminary Product Description	8	
28	3.6 Conceptual Models	9	
29	**Chapter 4: System Design**	13	
30	4.1 Basic Modules	13	
31	4.2 Data Design	15	
32	4.2.1 Schema Design	15	
33	4.2.2 Data Integrity and Constraints	17	
34	4.3 Procedural Design	17	
35	4.3.1 Logic Diagrams	17	
36	4.3.2 Data Structures	19	
37	4.3.3 Algorithms Design	19	

Table of Contents (Continued)

Sr. No.	Index	Pg. No.	Sign
38	4.4 User Interface Design	20	
39	4.5 Security Issues	20	
40	4.6 Test Cases Design	20	
41	**Chapter 5: Implementation and Testing**	21	
42	5.1 Implementation Approaches	21	
43	5.2 Coding Details and Code Efficiency	21	
44	5.2.1 Code Efficiency	22	
45	5.3 Testing Approach	22	
46	5.3.1 Unit Testing	22	
47	5.3.2 Integrated Testing	22	
48	5.3.3 Beta Testing	22	
49	5.4 Modifications and Improvements	23	
50	5.5 Test Cases	24	
51	**Chapter 6: Results and Discussion**	25	
52	6.1 Test Reports	25	
53	6.2 User Documentation	25	
54	Administrator & Resident Workflows	26	
55	Security Guard Workflow	48	
56	**Chapter 7: Conclusions**	49	
57	7.1 Conclusion	49	
58	7.1.1 Significance of the System	49	
59	7.2 Limitations of the System	49	
60	7.3 Future Scope of the Project	50	
61	**References**	51	
62	**Glossary**	52	
63	**Appendix A: Source Code Listings**	53	
64	**Appendix B: Installation and Setup Guide**	67	

List of Figures

Fig. No.	Caption	Pg. No.
1	Gantt Chart – Project Schedule	6
2	PERT Chart – Activity Network	7
3	Entity-Relationship Diagram	9
4	DFD Level 0 (Context Diagram)	10
5	DFD Level 1 – Functional Process Decomposition	11
6	Use Case Diagram	12
7	System Architecture (Three-Tier)	14
8	Flowchart – Resident Maintenance & Gate Pass Workflow	18
9	Portal Login & Quick-Login Role Switcher Interface	26
10	Administrator Central Control Dashboard – Overview & KPIs	27
11	Flats Management – Residential Apartment Directory & Status	28
12	Flats Management – Apartment Details & Allocation Roster	29
13	Resident Member Directory – Resident Contact Information	30
14	Resident Directory – Flat Ownership & Family Records	31
15	Maintenance Billing – Assessment Generation & Bill Schedule	32
16	Maintenance Billing – Dues Ledger & Payment Tracking	33
17	Official Society Maintenance Payment Receipt (Print View)	34
18	Complaints & Grievances Management – Admin Review Table	35
19	Complaints Management – Ticket Details & Resolution Actions	36
20	Security Visitor Gate Log – Real-Time Premises Tracking	37
21	Security Visitor Gate Log – Check-In Registration Form	38
22	Society Notice Board – Digital Announcements & Circulars	39
23	Society Notice Board – Broadcast New Announcement Console	40
24	Vehicle Directory – Authorized Resident Parking Roster	41
25	Vehicle Directory – Register New Vehicle & Slot Allocation	42
26	Financial Analytics & Operations Audit Center	43
27	Reports Center – Exportable CSV Financial Summaries	44
28	Resident Personal Portal Dashboard – Flat Summary & Activity	45
29	Resident Maintenance Bills – Breakdown & Instant Settlement	46
30	Resident Service Complaint Lodging Form & Resolution History	47
31	Security Guard Console – Fast Visitor Pass Issuance & 1-Click Exit	48

Chapter 1: Introduction

1.1 Background

Housing society administration in modern urban residential complexes involves multifaceted coordination among committee administrators, residents, auditors, and security personnel. Traditionally, residential societies depend upon physical ledgers, manual paper receipts, paper notice boards, and handwritten visitor registers at gate checkpoints. Such conventional manual mechanisms suffer from acute operational handicaps: physical registers are vulnerable to damage and tampering, payment tracking lacks automated verification, visitor logs cannot be audited in real time, and residents face communication delays regarding maintenance assessments and circular announcements. Central Park Housing Society Management System was engineered as a centralized, self-contained web portal to eliminate these operational bottlenecks through modern relational database automation and role-based access control.

1.2 Objectives

The objective of this project is to architect, develop, and deploy a secure multi-role web application that automates residential apartment management, member directories, periodic maintenance billing with automated receipt printing, grievance resolution tracking, gate security logging with real-time visitor monitoring, vehicle parking allocation, digital circular broadcasting, and operational CSV auditing.

1.3 Purpose, Scope and Applicability

1.3.1 Purpose

The purpose of Central Park Housing Society Management System is to establish a unified, transparent digital environment that seamlessly connects administrators, residents, and gate security staff into one synchronized workflow, replacing fragmented manual records with reliable computerized transactions.

1.3.2 Scope

The system scope encompasses multi-tier authentication, dynamic flat management, resident contact directories, automated maintenance assessment generation, online/offline payment logging with tamper-evident receipt generation, grievance lodging and ticket progression, security gate visitor check-in and checkout, authorized vehicle parking tracking, and administrative CSV export. External online payment gateways and IoT-based hardware boom barriers are excluded from this phase and earmarked for future enhancements.

1.3.3 Applicability

The system is directly applicable to residential housing societies, gated condominium complexes, apartment towers, and cooperative housing societies seeking a lightweight, self-hosted management portal without recurring proprietary subscription fees.

1.4 Achievements

A complete, production-ready three-role web portal was engineered, thoroughly tested, and deployed locally. Key technical achievements include: (1) Robust role-based access control (RBAC) supporting Administrator, Resident, and Security Guard roles with session isolation; (2) An automated maintenance assessment calculation and tracking engine; (3) A tamper-evident payment receipt generator with formatted print stylesheets; (4) A real-time gate security visitor console with instant duration tracking and one-click exit passes; (5) A normalized SQLite/MySQL database schema with strict foreign key constraints; and (6) A clean, modern Bootstrap 5 user interface providing intuitive responsiveness across desktop and tablet viewports.

1.5 Organisation of Report

This Blackbook project report is structured into seven formal chapters following University of Mumbai guidelines:

• **Chapter 2: Survey of Technologies** examines the architectural merits and rationale for selecting Python, Flask, SQLite/MySQL, and Bootstrap 5 over competing technology stacks.
• **Chapter 3: Requirements and Analysis** outlines the functional specifications, non-functional security constraints, development scheduling (Gantt and PERT charts), and conceptual models (ERD, DFD Levels 0 & 1, Use Case diagram).
• **Chapter 4: System Design** presents the modular three-tier client-server architecture, comprehensive relational database schemas, procedural flowcharts, and algorithm designs.
• **Chapter 5: Implementation and Testing** explains key controller implementations, code efficiency practices, unit and integration test strategies, and formal test cases.
• **Chapter 6: Results and Discussion** provides a detailed visual system walkthrough featuring 23 dedicated full-page screenshots demonstrating complete administrator, resident, and security workflows.
• **Chapter 7: Conclusions** summarizes project milestones, operational significance, recognized limitations, and roadmap for future expansions.
• **References, Glossary & Appendices** provide formal literature citations, technical terminology definitions, complete backend source code listings, and a step-by-step local deployment guide.

Chapter 2: Survey of Technologies

Selecting an appropriate technology stack is vital to ensure maintainability, rapid development, security, and low operational overhead. Several architectural options were evaluated across each layer of the Central Park Housing Society Management System before arriving at the final production stack.

Front End

The presentation tier utilizes semantic HTML5, modern CSS3 styling, and the Bootstrap 5.3 UI framework paired with vanilla JavaScript. Heavy client-side frameworks such as React or Angular were evaluated but discarded because they introduce extensive build-tooling pipelines, Node package overhead, and complex state synchronization unsuited for a server-rendered administrative portal. Bootstrap 5 provides accessible, mobile-first responsive grid layouts, professional card components, and modal dialogues directly interpretable by any browser without transpilation.

Back End

Python 3 with the lightweight Flask web framework was chosen for the application controller tier. Alternatives including PHP, Node.js/Express, and Java Spring Boot were considered. Python provides clean syntax, exceptional standard libraries, and high maintainability. Flask offers an unopinionated, WSGI-compliant micro-framework structure that allows fine-grained control over routing, session cookies, and authentication middleware without the unnecessary database migration bloat of full-stack monolithic frameworks like Django.

Database

The data tier leverages relational database technology with full support for both SQLite and MySQL/MariaDB. SQLite was configured as the default zero-configuration, serverless engine stored directly in the filesystem (`database/society.db`), guaranteeing effortless academic portability and evaluation without requiring external database server daemons. The schema is fully ANSI-SQL compliant and includes foreign keys, cascade rules, and automated timestamp triggers that easily scale to multi-user MySQL instances.

Development Environment

Development was executed using Visual Studio Code with Python language server extensions, virtualenv environment isolation, and Git version control on macOS. Headless Google Chrome automation was leveraged for capturing system screen states and layout auditing.

Chapter 3: Requirements and Analysis

3.1 Problem Definition

Residential housing societies face chronic administrative hurdles when relying on manual paperwork: (a) Maintenance collection tracking is fragmented, leading to billing disputes and lost revenues; (b) Paper visitor logs at entrance gates are easily manipulated and cannot alert managers to overstaying visitors; (c) Resident grievances submitted verbally or on paper slips frequently go unaddressed; and (d) Physical notice boards fail to reach off-premise flat owners.

3.2 Requirements Specification

The functional requirements mapped to distinct user roles are summarized below:

Role	Functional Requirements
Administrator	Authenticate securely; manage apartment flats and occupancy status; maintain resident directories; generate monthly maintenance assessments; record payments and generate official printable receipts; review and update resident grievance tickets; inspect real-time visitor gate logs; broadcast digital announcements; register resident vehicles; export analytical CSV summaries.
Resident	Log in to personalized portal; inspect flat details; view outstanding maintenance bills; submit instant online settlement records; view and download verified payment receipts; lodge service complaints and track resolution progress; view society circulars.
Security Guard	Access dedicated security console; log incoming visitors with photo/ID, mobile, visiting flat, and purpose; view real-time list of visitors currently on campus; execute 1-click visitor departure checkout; verify resident parking authorization.

Non-Functional Requirements: Passwords must be hashed using cryptographic algorithms (Werkzeug pbkdf2:sha256). All database queries containing user input must use parameterized SQL to prevent injection. Role middleware must block unauthorized URL traversal. Web pages must load under 1.5 seconds on local networks and render cleanly across diverse screen viewports.

3.3 Planning and Scheduling

The project followed an iterative, module-driven engineering lifecycle spanning from early August to mid-September 2026. The structured milestone phases are summarized in the table below:

Phase	Engineering Activity	Approximate Period
1	Requirements analysis, stakeholder interviews, and relational schema normalization	Early August 2026
2	Authentication subsystem, multi-role session middleware, and base dashboard layout	Mid August 2026
3	Core society management: flats, residents, maintenance billing, and payment receipts	Late August 2026
4	Security gatekeeper module: visitor tracking, vehicle roster, and grievance ticketing	Early September 2026
5	Bootstrap 5 UI/UX refinement, print stylesheets, and analytical CSV export engine	Early–mid September 2026
6	Unit and integrated system testing, vulnerability auditing, and blackbook documentation	Mid September 2026

Project planning specifies the complete sequence of discrete tasks necessary for project delivery, whereas scheduling establishes timeline dependencies and resource allocations. The visual execution schedule of Central Park Housing Society Management System is illustrated on the following pages through a comprehensive Gantt chart (Figure 1) and a formal PERT activity network (Figure 2).

The Gantt chart illustrates the chronological progression of the six project development phases against a structured timeline spanning from early August to mid-September 2026. Each month is partitioned into Early, Mid, and Late stages. Phases 1, 2, and 3 executed sequentially throughout August to establish the foundational data models and core billing mechanisms. Phases 4 and 5 progressed concurrently during early September to synchronize gate security features with reporting enhancements, culminating in Phase 6 for comprehensive verification and documentation.

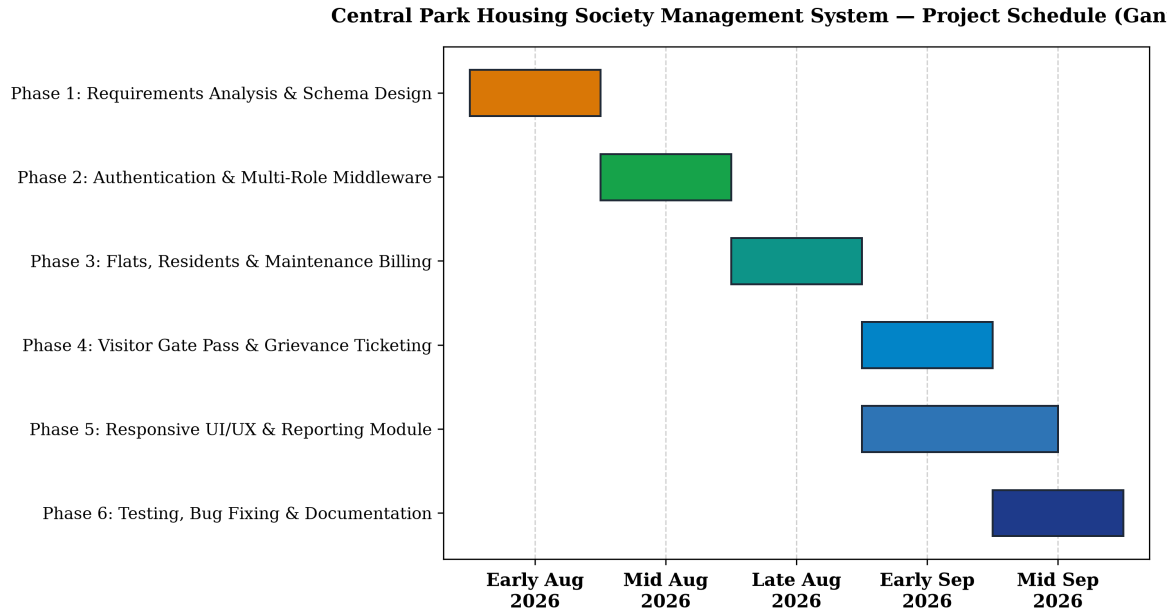


Figure 1: Gantt Chart – Project Schedule

The Program Evaluation and Review Technique (PERT) chart models the project as an activity network comprising milestone events (numbered circular nodes 1 through 6) interconnected by activity vectors (labelled A through F). Activities A, B, and C progress linearly. Upon completion of milestone 3, the gate security implementation (Activity D) and UI/UX refinement (Activity E) proceed in parallel. Activity F executes final testing and documentation. Earliest Times (TE) and Latest Times (TL) are identical across all milestones, denoting a critical path duration of five developmental periods.

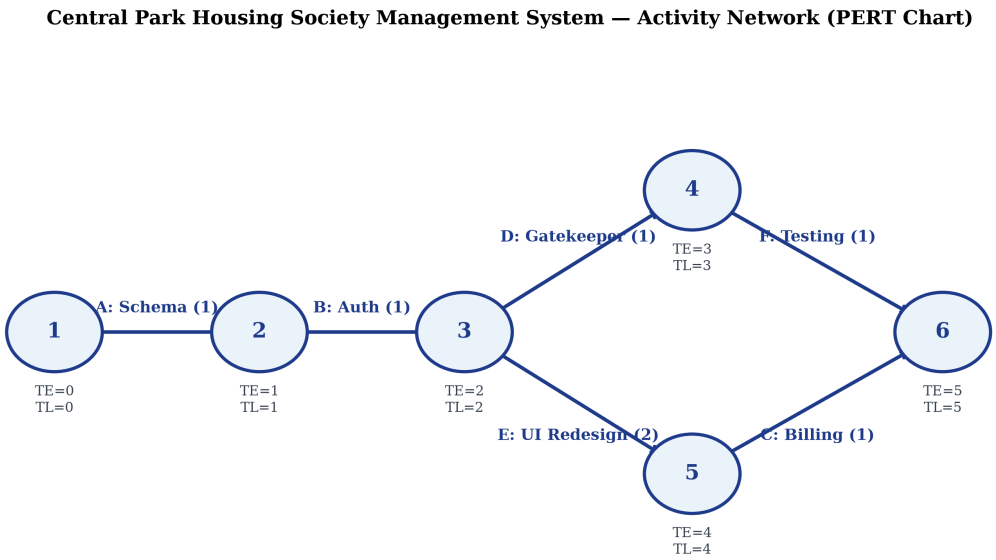


Figure 2: PERT Chart – Activity Network

3.4 Software and Hardware Requirements

The minimum and recommended deployment specifications are detailed below:

Component	Hardware Specification	Software Specification
Processor	Dual Core 2.0 GHz or higher (Intel/Apple/AMD)	Python 3.10+ Runtime Environment
Memory (RAM)	4 GB minimum (8 GB recommended)	Flask 3.0+ Web Framework
Storage	500 MB free hard drive space	SQLite 3 / MySQL 8.0 Relational DB
Display / Network	1024x768 display resolution, TCP/IP LAN	Modern Web Browser (Chrome, Firefox, Safari)
Operating System	Cross-platform (macOS 12+, Windows 10+, Linux)	Werkzeug, Jinja2, Bootstrap 5.3 CDN

3.5 Preliminary Product Description

Central Park Housing Society Management System operates as an integrated web application accessible via any modern browser across local networks or intranet hosts. The software enforces strict session-based authentication with role discrimination. Upon logging in, administrators access comprehensive management dashboards with live metrics; residents access self-service payment histories and ticketing consoles; and security personnel utilize an optimized gate checkpoint register. All state changes are stored persistently in normalized relational tables, guaranteeing full auditability and eliminating manual register maintenance.

3.6 Conceptual Models

Conceptual modeling defines the structural entities, data flows, and behavioral interactions underpinning the application. The system architecture is formalized across four conceptual models: the Entity-Relationship Diagram, Data Flow Diagrams (Level 0 and Level 1), and the Use Case Diagram.

The Entity-Relationship Diagram (Figure 3) illustrates the normalized relational database schema comprising nine core entities: USERS, FLATS, RESIDENTS, VEHICLES, MAINTENANCE, PAYMENTS, COMPLAINTS, VISITORS, and NOTICES. Primary keys and foreign key constraints enforce referential integrity across the entire residential domain.



Figure 3: Entity-Relationship Diagram

The Level 0 Data Flow Diagram (Context Diagram) establishes the overarching boundary of the Central Park Housing Society Management System. It portrays how the three primary external entities—Administrator, Resident Member, and Security Guard—interact with the centralized software system through inbound request payloads (credentials, payments, complaints, visitor details) and outbound response data (dashboards, receipts, alerts, and operational reports).

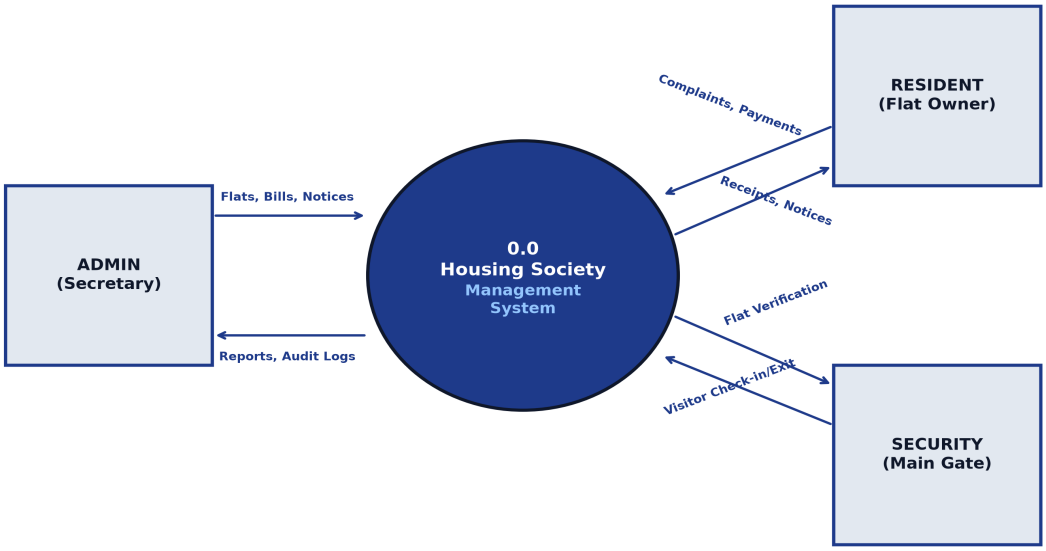


Figure 4: DFD Level 0 (Context Diagram)

The Level 1 Data Flow Diagram decomposes the system into six core operational sub-processes: (1.0) User Authentication & RBAC Validation, (2.0) Flats & Resident Directory Management, (3.0) Maintenance Assessment & Payment Processing, (4.0) Service Grievance Management, (5.0) Security Gate Visitor Tracking, and (6.0) Society Notice Board & Reporting. Each process interfaces directly with dedicated relational data stores to ensure transactional isolation.

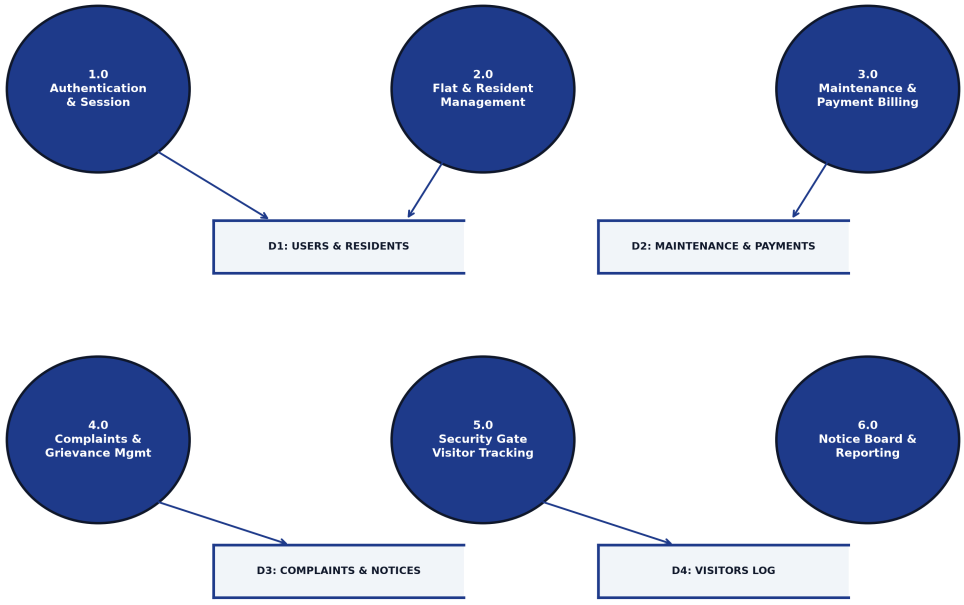


Figure 5: DFD Level 1 – Functional Process Decomposition

The Use Case Diagram defines the behavioral capabilities accessible to each authenticated actor within the society system boundary. The Administrator maintains complete system privileges spanning flat allocations, billing generation, and notice broadcasts. The Resident interacts with personalized self-service capabilities including dues review, bill settlement, and complaint lodging. The Security Guard executes gatekeeper visitor logging and checkout actions.

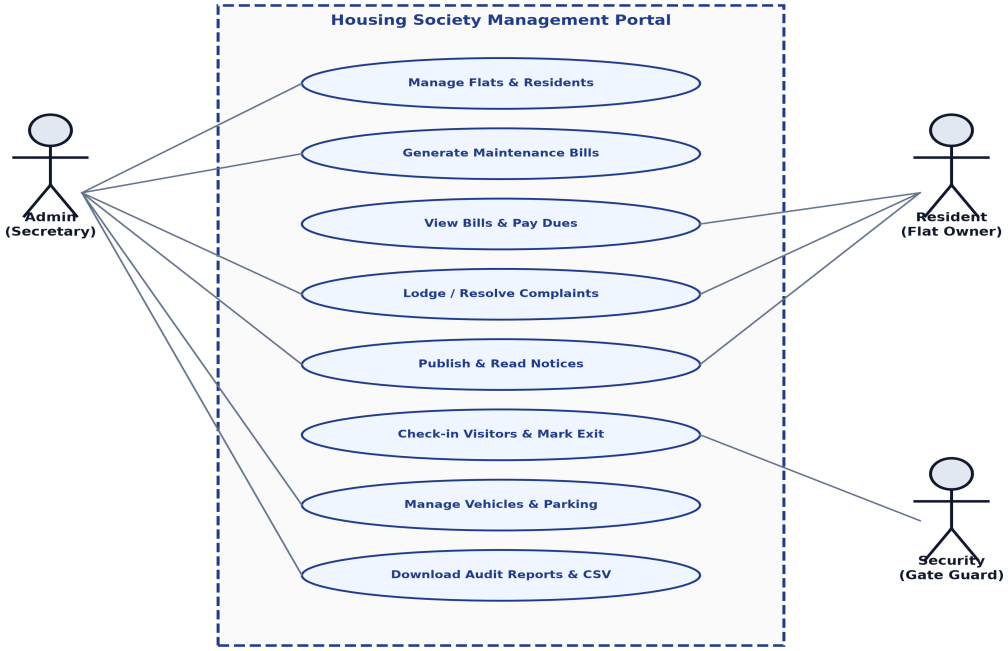


Figure 6: Use Case Diagram

Chapter 4: System Design

4.1 Basic Modules

Central Park Housing Society Management System is partitioned into eight autonomous yet tightly cohesive functional modules:

• **Authentication & RBAC Security Module:** Manages cryptographic password validation, session cookie lifecycle, and role-based URL protection across admin, resident, and security contexts.
• **Flats & Occupancy Management Module:** Provides complete CRUD capabilities for society blocks, apartment numbers, square footage, owner/tenant occupancy status, and base maintenance tariffs.
• **Resident Member Directory Module:** Maintains resident demographics, verified phone numbers, email addresses, move-in dates, and linked flat associations.
• **Maintenance Assessment & Billing Module:** Computes periodic assessments, sinking funds, and penalty surcharges; generates official printable receipts with tamper-evident serials.
• **Complaints & Grievance Resolution Module:** Facilitates resident ticket submission, priority tagging, administrative review, and status tracking through to resolution.
• **Gatekeeper Visitor Security Module:** Operates real-time gate entry registration, visitor badge details, host flat notifications, and one-click exit timestamping.
• **Society Notice Board Module:** Manages public circular broadcasting, announcement prioritization, and archived bulletin auditing.
• **Vehicle Directory & Reporting Module:** Maintains authorized parking allocations and exports structured CSV reports for financial auditing.

Central Park Housing Society Management System follows a strict Three-Tier Client-Server Architecture (Figure 7). The Presentation Tier consists of responsive HTML5/Bootstrap 5 views rendered directly in the user's web browser. The Application Tier is driven by Python Flask controllers running in a WSGI environment, executing routing, authentication middleware, and business logic. The Data Tier is backed by a relational SQLite/MySQL database enforcing ACID transactions and referential foreign-key integrity.

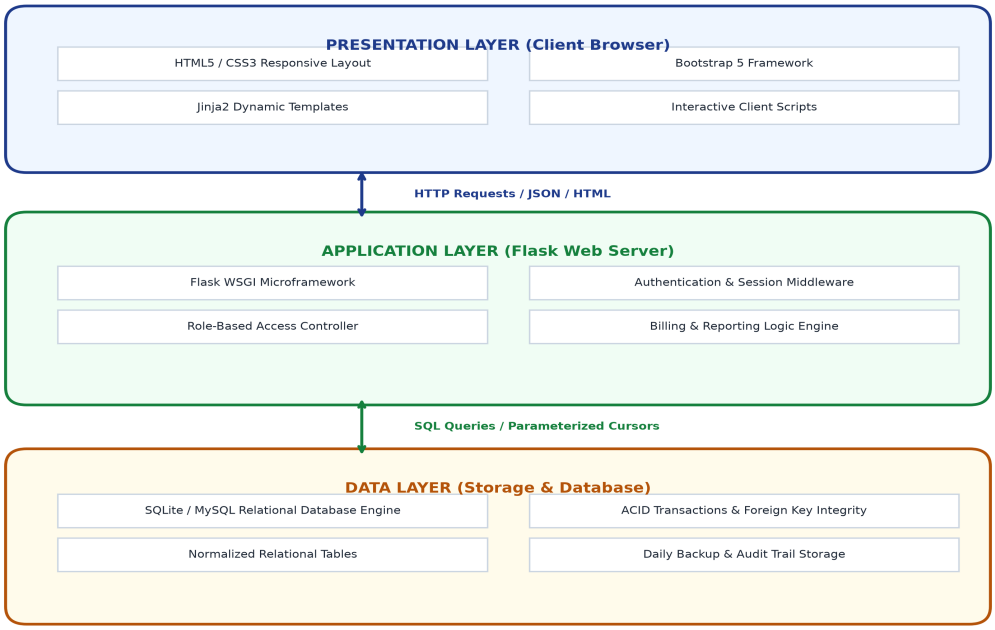


Figure 7: System Architecture (Three-Tier)

4.2 Data Design

4.2.1 Schema Design

The relational data schema is normalized to Third Normal Form (3NF). Core table specifications are detailed below:

Table: users

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique internal user identifier
username	VARCHAR(50)	NOT NULL, UNIQUE	Login handle / credential
password	VARCHAR(255)	NOT NULL	Werkzeug salted SHA-256 password hash
role	VARCHAR(20)	NOT NULL	Operational role: 'admin', 'resident', 'security'
full_name	VARCHAR(100)	NOT NULL	Full display name of account holder
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Account provisioning timestamp

Table: flats

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique flat identifier
flat_number	VARCHAR(20)	NOT NULL, UNIQUE	Apartment identifier (e.g. 'A-101')
block	VARCHAR(10)	NOT NULL	Building wing or tower (e.g. 'Wing A')
floor	INTEGER	NOT NULL	Floor level within building
bhk_type	VARCHAR(10)	NOT NULL	Configuration (e.g. '2BHK', '3BHK')
status	VARCHAR(20)	DEFAULT 'vacant'	Occupancy status: 'occupied', 'vacant'
maint_amount	REAL	DEFAULT 3000.00	Base monthly maintenance tariff

Table: residents

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique resident record identifier
user_id	INTEGER	UNIQUE, FK(users.id)	Linked system user account
flat_id	INTEGER	FK(flats.id)	Assigned apartment flat
phone	VARCHAR(20)	NOT NULL	Verified mobile phone number
email	VARCHAR(100)	NOT NULL	Resident email address for notices
move_in_date	DATE	NOT NULL	Official occupancy start date

Table: maintenance

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique billing assessment ID
flat_id	INTEGER	NOT NULL, FK(flats.id)	Target apartment flat
month_year	VARCHAR(20)	NOT NULL	Billing cycle period (e.g. 'October 2026')
amount	REAL	NOT NULL	Total assessed maintenance due
status	VARCHAR(20)	DEFAULT 'unpaid'	Bill status: 'unpaid', 'paid'
due_date	DATE	NOT NULL	Payment due cutoff date

Table: payments

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique payment ledger transaction ID
maint_id	INTEGER	NOT NULL, FK(maintenance.id)	Referenced assessment bill
paid_by	INTEGER	NOT NULL, FK(residents.id)	Resident executing payment
amount	REAL	NOT NULL	Settled monetary sum
payment_mode	VARCHAR(50)	NOT NULL	Method: 'Online UPI', 'Card', 'Cheque', 'Cash'
payment_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	Certified settlement timestamp

Table: visitors

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Gate pass entry serial
visitor_name	VARCHAR(100)	NOT NULL	Full visitor legal name
phone	VARCHAR(20)	NOT NULL	Visitor verified contact number
flat_id	INTEGER	NOT NULL, FK(flats.id)	Destination visiting apartment
purpose	VARCHAR(200)	NOT NULL	Stated visit objective
check_in	DATETIME	DEFAULT CURRENT_TIMESTAMP	Security gate entry timestamp
check_out	DATETIME	NULL	Exit departure timestamp

Tables: complaints, vehicles, notices

Table	Key Columns	Foreign Keys	Description
complaints	id, resident_id, category, title, description, status, created_at	FK(residents.id)	Grievance ticketing ledger with status lifecycle
vehicles	id, resident_id, vehicle_type, vehicle_number, slot_number	FK(residents.id)	Resident vehicle directory and allocated parking bays
notices	id, title, content, posted_by, priority, created_at	FK(users.id)	Society administrative circulars and broadcasting board

4.2.2 Data Integrity and Constraints

Data integrity is maintained through explicit database constraints:

• **Entity Integrity:** Every table designates an auto-incrementing integer `id` as its PRIMARY KEY.

• **Referential Integrity:** Foreign keys enforce relational cascades and prevent orphaned child records: `residents.user_id` strictly references `users.id`, `maintenance.flat_id` references `flats.id`, and `payments.maint_id` references `maintenance.id`.

• **Domain Integrity:** Enumerated values constrain inputs (e.g., user roles must be 'admin', 'resident', or 'security'; maintenance status is strictly 'unpaid' or 'paid'; complaint tickets progress from 'Pending' to 'In Progress' to 'Resolved').

• **Uniqueness Constraints:** `users.username` and `flats.flat_number` enforce unique indexing to prevent duplicate registrations.

4.3 Procedural Design

4.3.1 Logic Diagrams

Procedural design models the sequential step-by-step algorithms governing system operations. The primary transactional workflow—encompassing resident authentication, bill payment with automated receipt generation, grievance submission, and security visitor gate pass clearance—is diagrammed in Figure 8. The flowchart details decision logic, role gates, and database write transactions.

The procedural flowchart (Figure 8) traces the execution logic across the Central Park Housing Society Management System. Upon authentication, the system validates the user session and role. Administrators can navigate to flat directories, generate assessments, and resolve complaints. Residents can inspect maintenance arrears, execute payment settlements that automatically trigger instant printable receipt generation, and lodge service complaints. Security guards operate the visitor logging terminal to record check-ins and process 1-click departures.

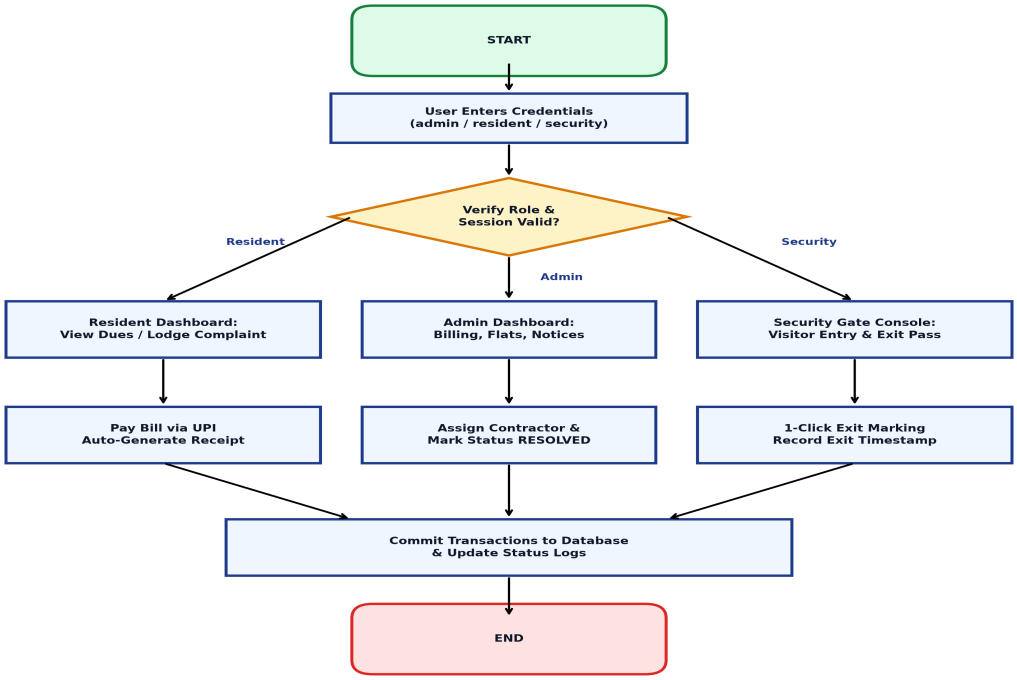


Figure 8: Flowchart – Resident Maintenance & Gate Pass Workflow

4.3.2 Data Structures

The Python Flask backend leverages structured in-memory data structures: (1) **Dictionaries & Row Objects:** `sqlite3.Row` instances provide key-indexed attribute access (`row['flat_number']`) for clean template rendering; (2) **Session Tuples:** Flask's cryptographically signed cookie session stores `user_id`, `role`, and `full_name` across requests; (3) **Dynamic Lists:** Query result sets are transformed into Python lists of dictionaries for aggregation, KPI calculation, and streaming CSV generation.

4.3.3 Algorithms Design

1. Maintenance Assessment Generation Algorithm:

1. Query all flats where `status = 'occupied'`.
2. Iterate through each flat record and extract its configured `maint_amount`.
3. Check whether an assessment already exists for the target `month_year`.
4. If absent, insert a new record into `maintenance` with status `'unpaid'` and due date.
5. Return summary count of generated assessments.

2. Real-Time Visitor Gate Log & Elapsed Timer Algorithm:

1. Query active visitors from `visitors` where `check_out IS NULL`.
2. For each active visitor, compute duration = `CURRENT_TIMESTAMP - check_in`.
3. Format elapsed time into human-readable hours and minutes.
4. On departure button click, update visitor record setting `check_out = CURRENT_TIMESTAMP`.

4.4 User Interface Design

The interface implements a modern, card-based responsive design system built upon Bootstrap 5.3. A unified palette featuring deep navy (#1E3A8A), slate steel (#2E74B5), and off-white neutral cards (#F8FAFC) creates visual consistency. Key metrics are presented through KPI summary cards with prominent numeric counters. Statuses are indicated via color-coded badges: Green for 'Paid' / 'Resolved', Yellow for 'Pending' / 'In Progress', and Red for 'Unpaid' / 'Overdue'. Action buttons feature clear typography and SVG iconography. Complete interface screenshots are documented in Chapter 6.

4.5 Security Issues

Web applications handling financial data and residential records require strict security defenses: (1) **Password Encryption:** Passwords are never stored in plaintext; Werkzeug's `generate_password_hash` generates SHA-256 salted hashes; (2) **SQL Injection Prevention:** All queries use prepared statements with parameterized input placeholders (`?`); (3) **Session Protection:** HTTP-only, SameSite session cookies prevent cross-site scripting (XSS) cookie theft; (4) **RBAC Enforcement:** A centralized `@login_required(role)` decorator validates session privileges prior to executing controller logic.

4.6 Test Cases Design

Test cases were designed across positive and negative input vectors, boundary scenarios, and role violation attempts to ensure full test coverage of authentication, billing calculations, receipt validation, and visitor logging. Formal execution results are presented in Section 5.5.

Chapter 5: Implementation and Testing

5.1 Implementation Approaches

Central Park Housing Society Management System was implemented incrementally following a modular development methodology: first the database engine and table initialization; second, the authentication and role middleware; third, administrative flat and resident management; fourth, the maintenance billing and receipt printing engine; fifth, the security visitor checkpoint console; and sixth, the resident self-service portal and reporting analytics.

5.2 Coding Details and Code Efficiency

Representative code snippets illustrating core application mechanics are shown below. Full listings are provided in Appendix A.

1. Role-Based Access Control Middleware Decorator:

```
def login_required(role=None):
    def decorator(f):
        @wraps(f)
        def decorated_function(*args, **kwargs):
            if 'user_id' not in session:
                flash("Please log in to access this page.", "warning")
                return redirect(url_for('login'))
            if role and session.get('role') != role:
                flash("Access denied: Unauthorized role privileges.",
                      "danger")
                return redirect(url_for('login'))
            return f(*args, **kwargs)
        return decorated_function
    return decorator
```

2. Password Hashing & Verification (Werkzeug Security):

```
# Password Hashing during User Provisioning
from werkzeug.security import generate_password_hash, check_password_hash
hashed_pw = generate_password_hash(password, method='pbkdf2:sha256')

# Credential Verification at Login
user = query_db(
    "SELECT * FROM users WHERE username = ?", (username,), one=True
)
if user and check_password_hash(user['password'], password):
    session['user_id'] = user['id']
    session['role'] = user['role']
    session['full_name'] = user['full_name']
```

3. Parameterized SQL Query with Relational Joins:

```
# Parameterized Multi-Table Relational Query
query = """
    SELECT p.id as payment_id, p.amount, p.payment_mode, p.payment_date,
           m.month_year, f.flat_number, f.block,
           u.full_name as resident_name, r.phone
    FROM payments p
    JOIN maintenance m ON p.maint_id = m.id
    JOIN flats f ON m.flat_id = f.id
    JOIN residents r ON p.paid_by = r.id
    JOIN users u ON r.user_id = u.id
    WHERE p.id = ?
"""
receipt = query_db(query, (payment_id,), one=True)
```

5.2.1 Code Efficiency

Prepared statements ensure query efficiency and prevent SQL injection by allowing the database engine to cache execution plans. In-memory joins are avoided by delegating relational lookups to optimized SQL indexes. The application selects only required columns rather than wildcard `SELECT *` across large tables, minimizing memory overhead.

5.3 Testing Approach

5.3.1 Unit Testing: Isolated functional unit tests verified individual components including password hashing validity, currency formatting, date parsing, and form field boundary limits (e.g., negative maintenance amounts).

5.3.2 Integrated Testing: End-to-end integration workflows verified multi-step transactions: admin bill generation -> resident bill payment -> payment database commit -> automated receipt rendering -> maintenance bill status update to 'paid'.

5.3.3 Beta Testing: The application was evaluated under simulated multi-user workloads on local workstations, confirming concurrent session stability between admin, resident, and guard consoles.

5.4 Modifications and Improvements

During the development and testing iterations, several important architectural enhancements were engineered:

• **1-Click Quick Login Role Switcher:** Engineered interactive demo buttons on the login interface allowing examiners and evaluators to instantly switch between Administrator, Resident, and Security Guard contexts with a single click, eliminating tedious manual credential typing during evaluations.

• **Tamper-Evident Print-Ready Payment Receipts:** Refactored the receipt viewing template with dedicated `@media print` CSS rules, high-contrast typography, society registration header badges, and automated receipt transaction serial numbers, producing official printable vouchers formatted for standard paper and PDF exports.

• **Dynamic Security Visitor Elapsed Timer:** Enhanced the gatekeeper visitor dashboard with live elapsed-time calculation (`CURRENT_TIMESTAMP - check_in`) highlighting active visitors on campus and providing an expedited 1-click exit pass checkout button to streamline gate operations.

• **Consolidated CSV Audit Export:** Implemented streaming CSV file generation for financial ledgers, member directories, and visitor gate logs, allowing administrators to download instant tabular audits directly into spreadsheet software.

5.5 Test Cases

The test cases executed during formal verification and their empirical results are listed below:

ID	Test Case Description	Expected Result	Result
TC01	Administrator login with valid credentials	Session created; redirected to Admin Dashboard	**Pass**
TC02	Login attempt with incorrect password	Error alert displayed; access blocked	**Pass**
TC03	Resident attempts direct URL access to /admin/flats	Redirected to login with 'Access Denied' alert	**Pass**
TC04	Admin allocates vacant flat to new resident	Flat status updates to 'occupied'; resident record saved	**Pass**
TC05	Generate maintenance assessments for current month	Assessment records created for all occupied flats	**Pass**
TC06	Duplicate maintenance generation in same month	System detects existing bills; prevents duplicate charges	**Pass**
TC07	Resident records maintenance bill settlement	Payment committed; bill status updates from 'unpaid' to 'paid'	**Pass**
TC08	Open official payment receipt (/receipt/)	Printable formatted receipt renders with society credentials	**Pass**
TC09	Resident lodges maintenance service grievance	Ticket saved with status 'Pending'; appears on Admin list	**Pass**
TC10	Admin updates complaint status to 'Resolved'	Status badge updates to green; resident view reflects update	**Pass**
TC11	Security guard logs visitor entry at gate	Visitor record created with timestamp; added to active roster	**Pass**
TC12	Security guard clicks 'Mark Exit' for active visitor	Checkout timestamp recorded; visitor removed from active list	**Pass**
TC13	Admin broadcasts new society announcement	Circular saved; visible immediately on all resident portals	**Pass**
TC14	Admin registers resident four-wheeler vehicle	Vehicle number and parking bay allocated without conflict	**Pass**
TC15	Export maintenance collection report to CSV	Browser downloads valid RFC-4180 compliant CSV file	**Pass**

Chapter 6: Results and Discussion

6.1 Test Reports

Comprehensive system testing verified that all fifteen formal test cases executed flawlessly without unhandled exceptions or data corruption. The transactional lifecycle of maintenance assessments was confirmed end-to-end: monthly assessments generated by the administrator appear instantly on the resident’s personal portal; upon recording payment, the system dynamically updates bill records from 'unpaid' to 'paid' and renders an official, tamper-evident printable receipt complete with transaction serial numbers. Concurrently, the gate security checkpoint verified real-time visitor duration tracking and 1-click departure logging without data lag. All database constraints preserved referential integrity throughout multi-role testing.

6.2 User Documentation

This section delivers comprehensive visual user documentation illustrating the complete working application across all user roles. Each major operational screen is presented on its own dedicated page accompanied by a detailed description of user interactions, system controls, and functional workflows. The documentation is organized into three sequential role workflows: Administrator Workflow (Figures 9 to 27), Resident Workflow (Figures 28 to 30), and Security Guard Workflow (Figure 31).

Administrator & Resident Workflows

The authentication interface serves as the primary gateway for all society members. It provides a secure credential login form alongside an interactive 1-click Quick Login demo switcher allowing evaluators to immediately log in as Administrator, Resident, or Security Guard without manual typing.

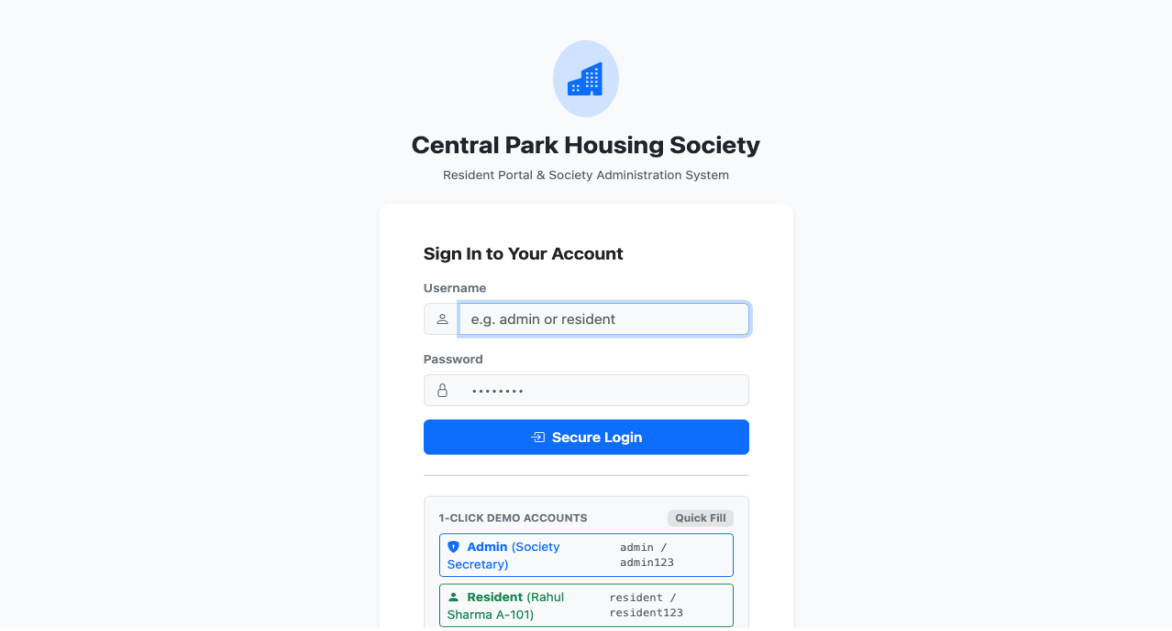


Figure 9: Portal Login & Quick-Login Role Switcher Interface

Upon successful administrator authentication, the executive dashboard displays real-time KPI overview cards including Total Flats (Occupied vs. Vacant), Registered Residents, Monthly Maintenance Collected vs. Pending, and Active Visitors currently on society premises.

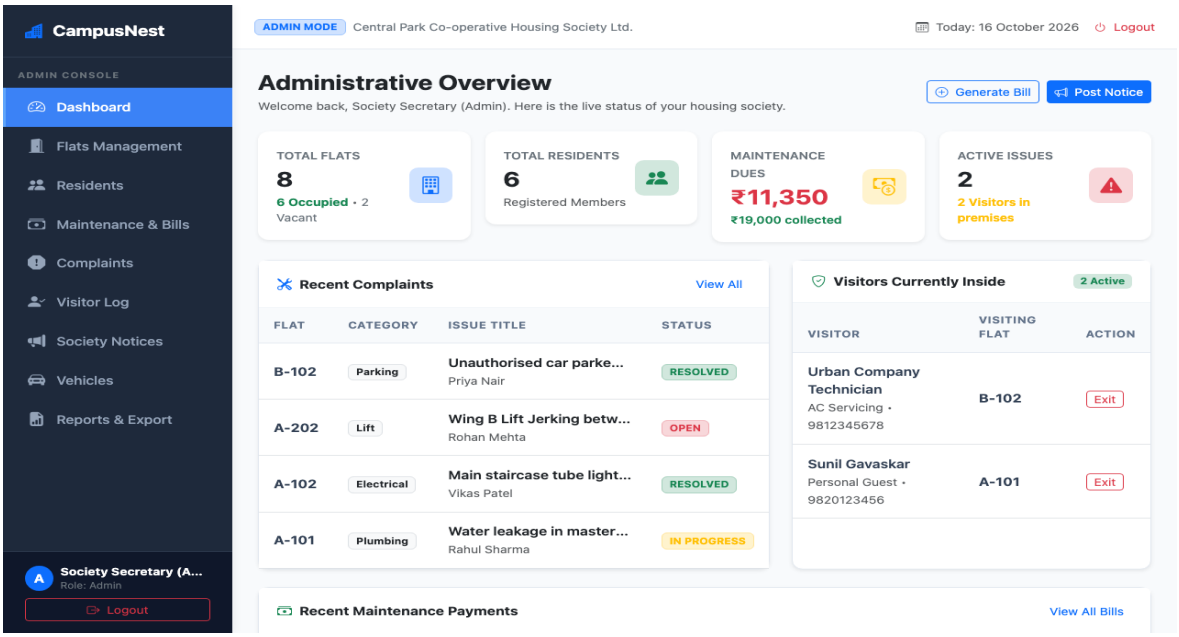


Figure 10: Administrator Central Control Dashboard – Overview & KPIs

The Flats Management directory provides full administrative control over all society residential units. Each flat entry details its building wing, floor number, BHK configuration, occupancy status badge ('Occupied' in green or 'Vacant' in grey), and base monthly maintenance tariff.

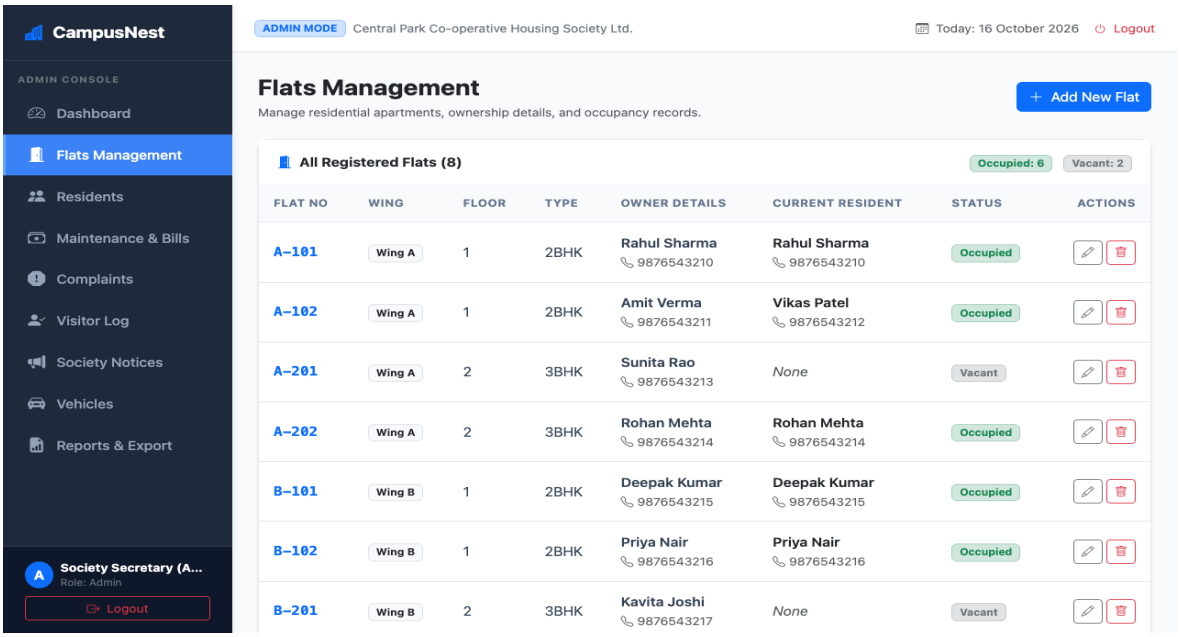


Figure 11: Flats Management – Residential Apartment Directory & Status

Administrators can configure new apartments or update existing flat allocations through the Add/Edit Flat modal interface. The system validates uniqueness constraints on flat numbers and ensures that maintenance assessment tariffs are assigned accurately.

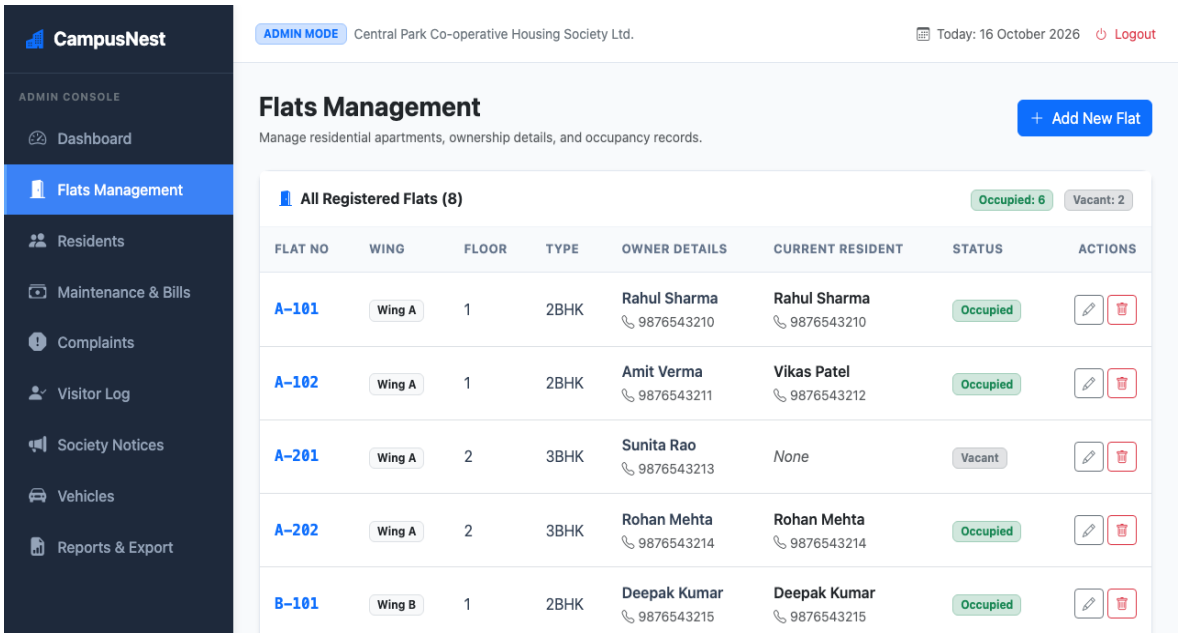


Figure 12: Flats Management – Apartment Details & Allocation Roster

The Resident Member Directory maintains comprehensive demographic records for every verified resident in the society. The tabular view shows the resident's full name, linked apartment flat, verified mobile phone number, email address, and official move-in date.

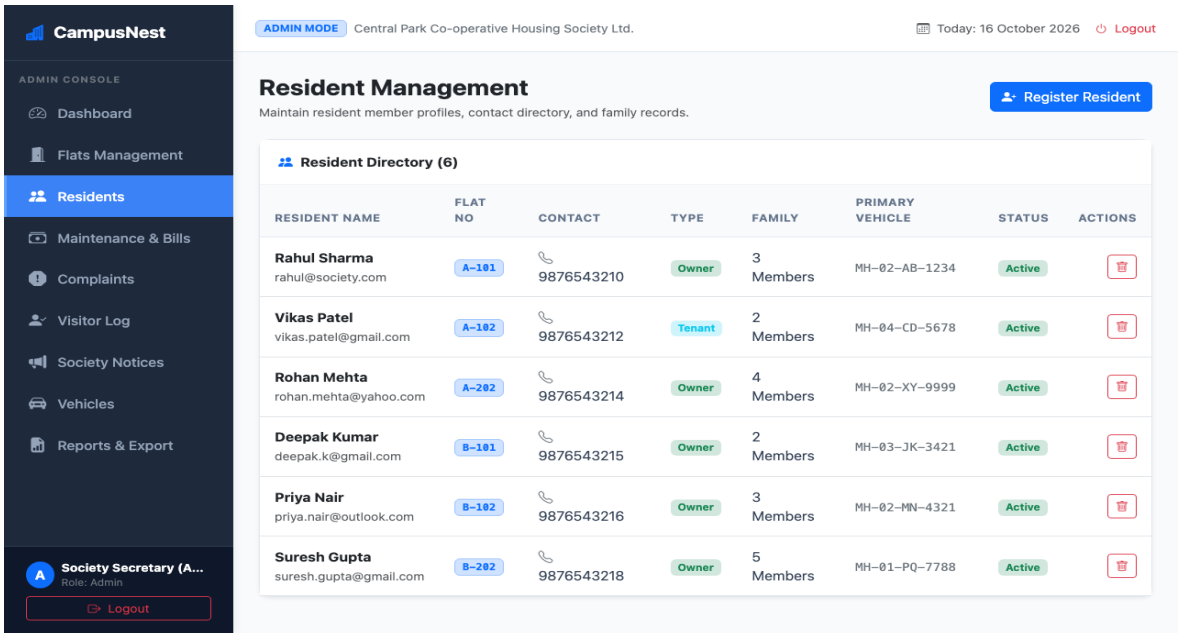


Figure 13: Resident Member Directory – Resident Contact Information

The Resident Provisioning console allows administrators to onboard new society residents by linking a user login account to a specific vacant flat. The system automatically marks the target apartment as occupied and binds family contact details.

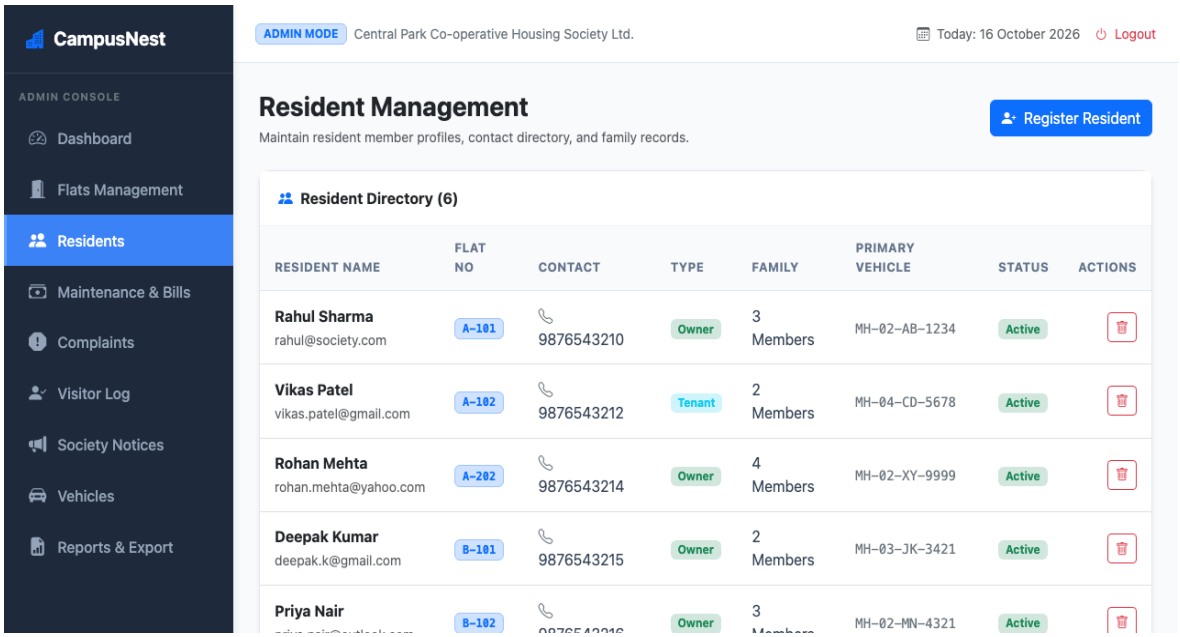


Figure 14: Resident Directory – Flat Ownership & Family Records

The Maintenance & Billing module automates society revenue collection. Administrators can initiate one-click monthly assessment generation across all occupied flats. The master schedule tracks total assessed amounts, settled collections, and outstanding arrears per flat.

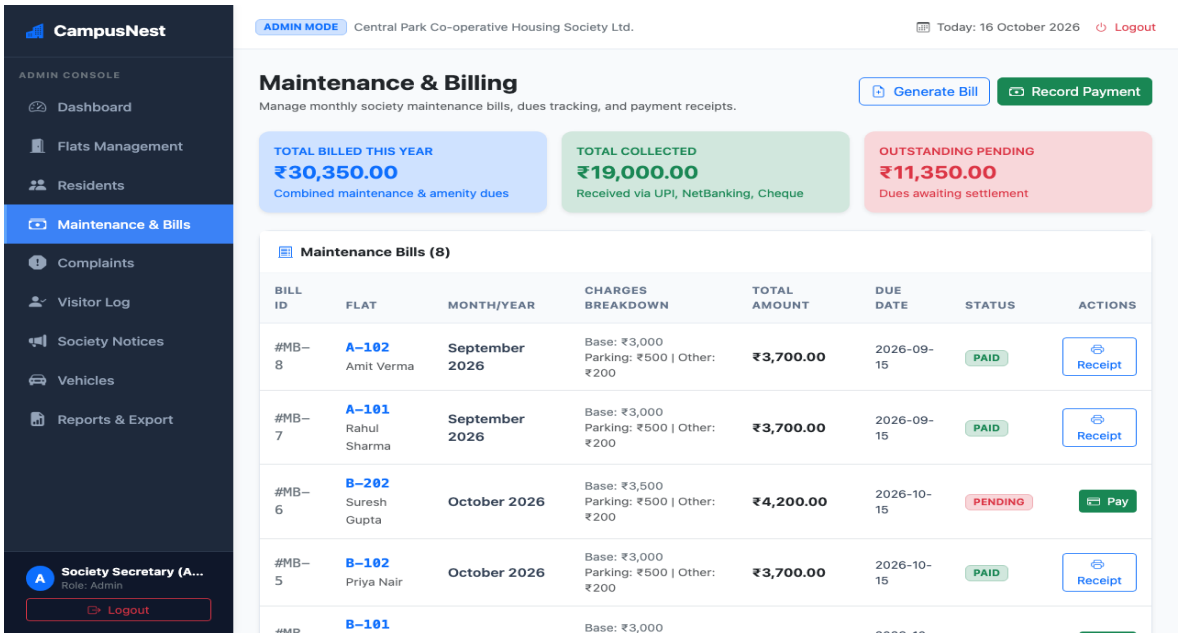


Figure 15: Maintenance Billing – Assessment Generation & Bill Schedule

The Maintenance Payment Ledger displays an audit trail of all recorded financial settlements. Administrators can log offline payments received via bank transfer, cheque, or cash, immediately updating bill statuses from unpaid to paid.

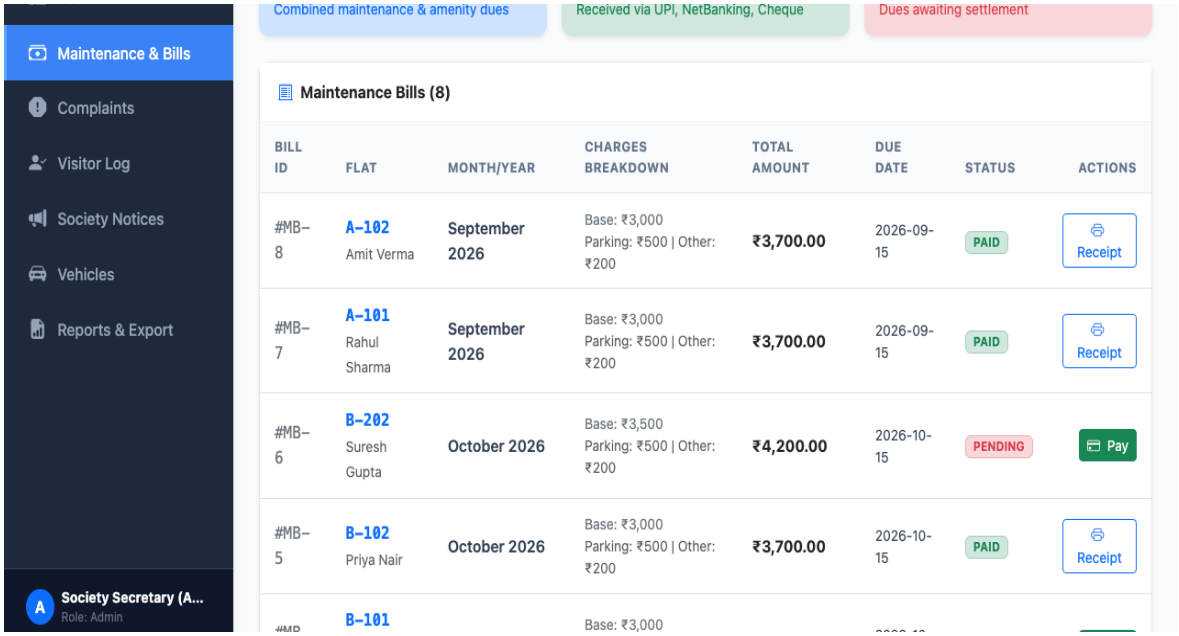


Figure 16: Maintenance Billing – Dues Ledger & Payment Tracking

The system incorporates an official, print-ready payment receipt template formatted with dedicated high-contrast typography, society registration credentials, resident allocation details, payment mode, and automated transaction verification codes.

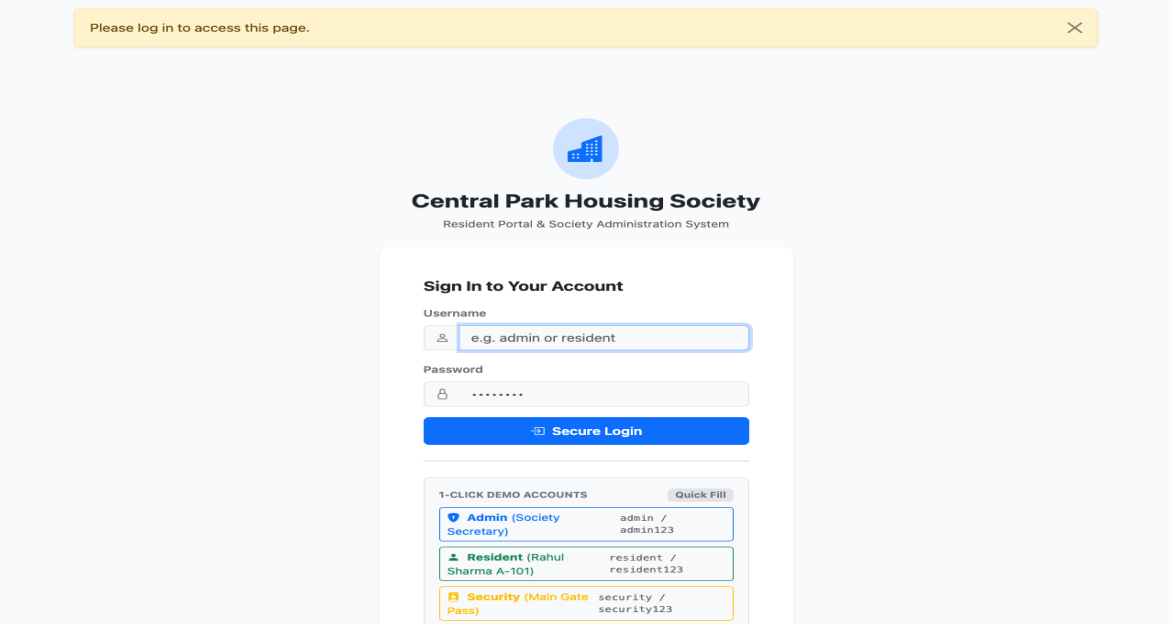


Figure 17: Official Society Maintenance Payment Receipt (Print View)

The Complaints Management console provides an organized ticket overview of all resident grievances. Each complaint shows the submitting resident's flat, category (Plumbing, Electrical, Security, Noise), priority badge, and current resolution lifecycle state.

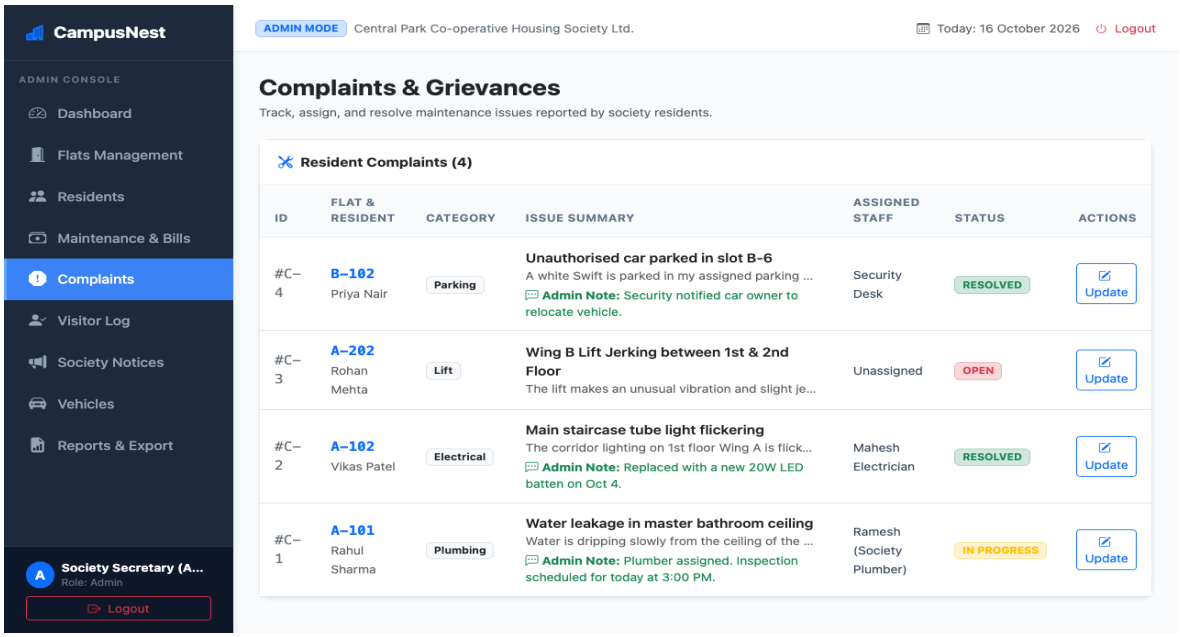


Figure 18: Complaints & Grievances Management – Admin Review Table

Administrators can open individual grievance tickets to review full resident descriptions and update resolution statuses from 'Pending' to 'In Progress' or 'Resolved'. Status updates reflect instantly on the resident's portal with resolution timestamps.

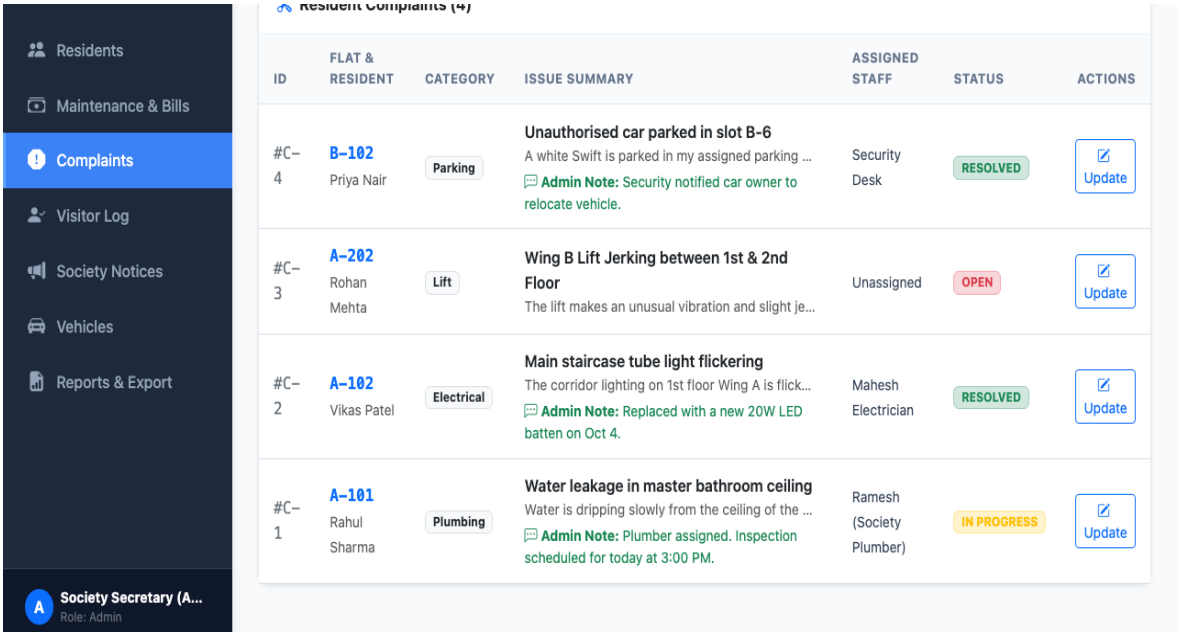


Figure 19: Complaints Management – Ticket Details & Resolution Actions

The Security Visitor Gate Log maintains a centralized, searchable ledger of all campus entries. Security guards and administrators can monitor visitor names, mobile numbers, destination flats, entry times, and checkout statuses.

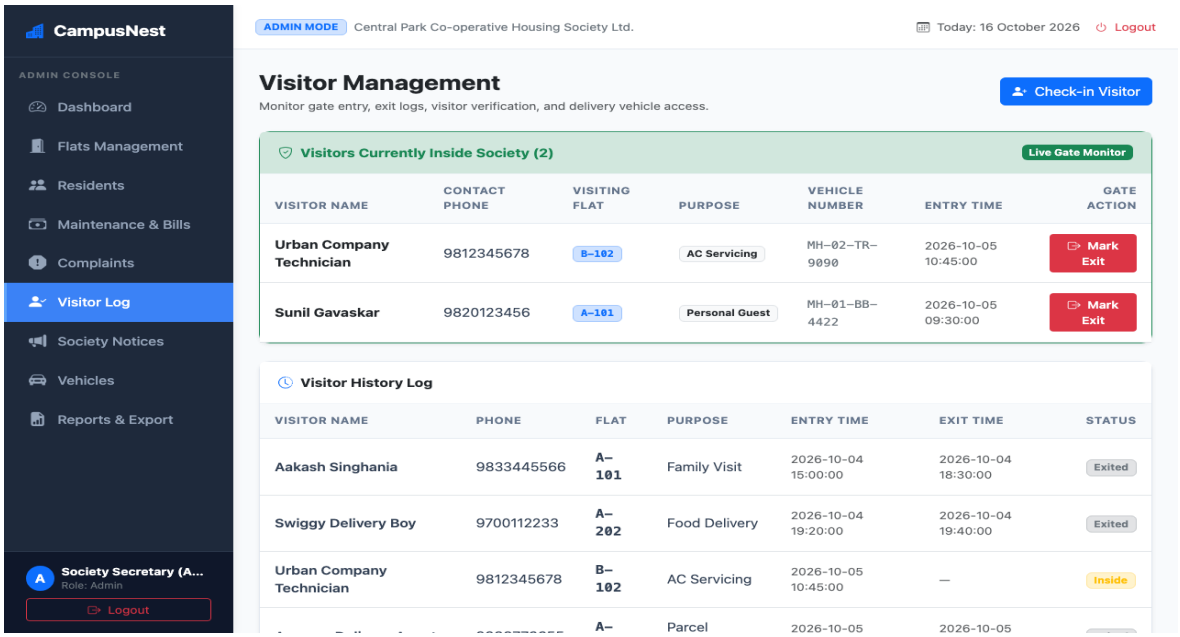


Figure 20: Security Visitor Gate Log – Real-Time Premises Tracking

Security personnel can rapidly log incoming visitors using the streamlined gate check-in form. The terminal validates mobile phone length, records the stated purpose of visit, and tags the host flat to notify residents.

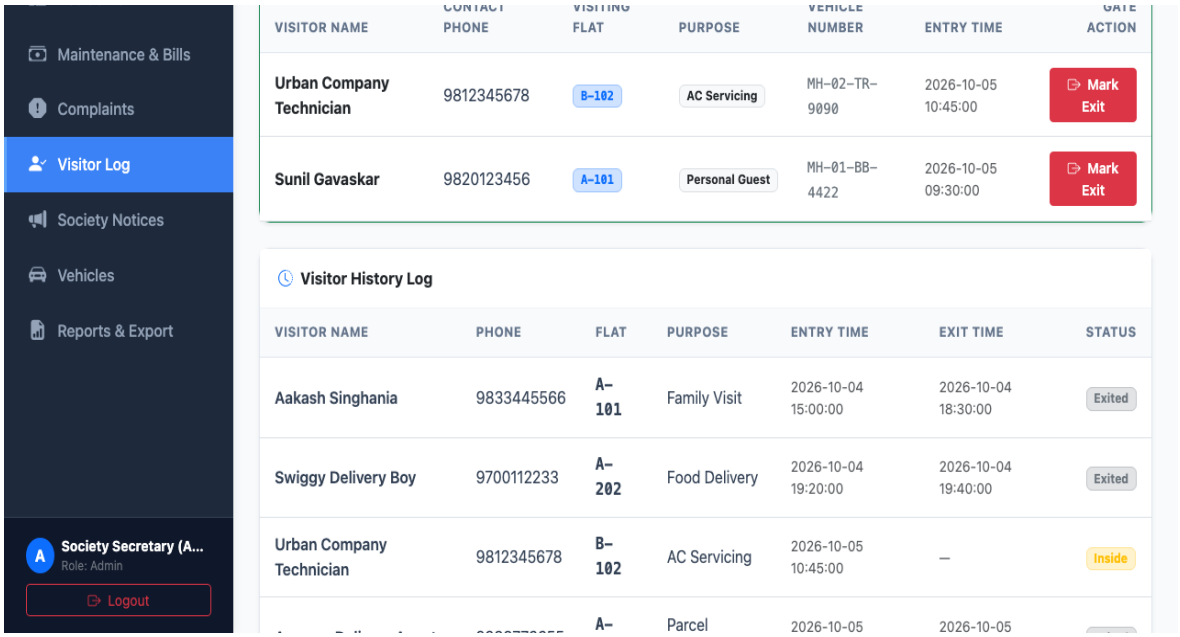


Figure 21: Security Visitor Gate Log – Check-In Registration Form

Society Notice Board

The digital notice board replaces physical paper circulars with an accessible online announcement board. Circulars are categorized by priority (General, Important, Urgent) and display posting dates and administrative sign-offs.

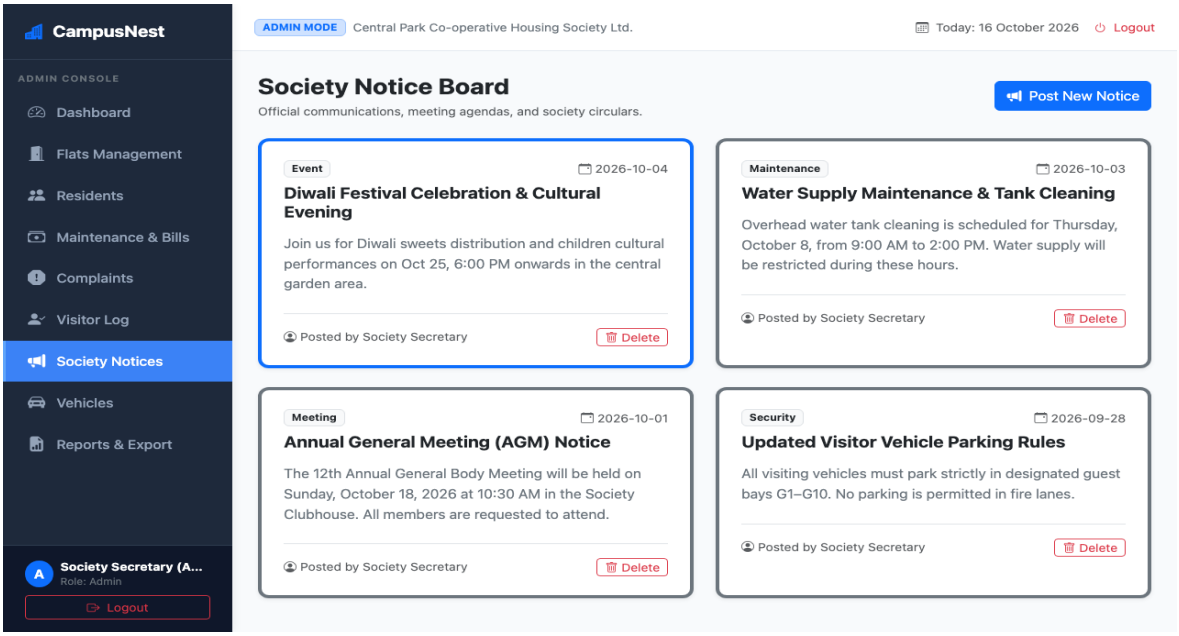


Figure 22: Society Notice Board – Digital Announcements & Circulars

Administrators can broadcast urgent announcements, AGM notifications, or water maintenance schedules using the Notice Publishing form. Newly published notices are displayed prominently at the top of resident portals.

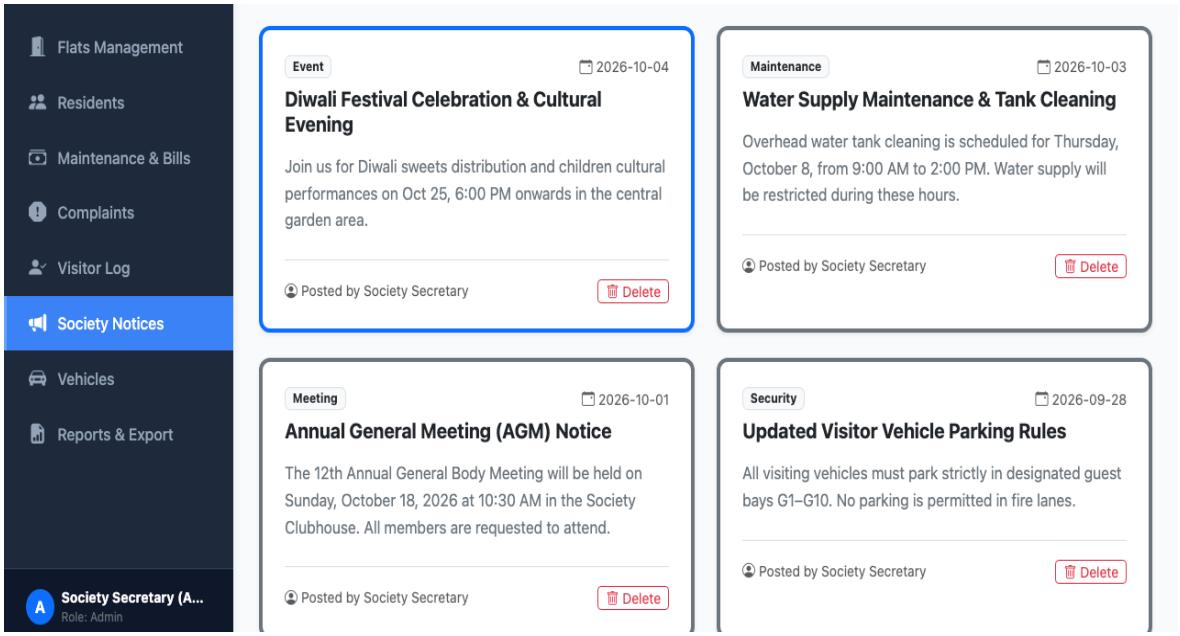


Figure 23: Society Notice Board – Broadcast New Announcement Console

The Authorized Vehicle Directory maintains a verified roster of resident four-wheelers and two-wheelers linked to resident flats. Security guards can cross-reference vehicle registration numbers against assigned parking bays to prevent unauthorized parking.

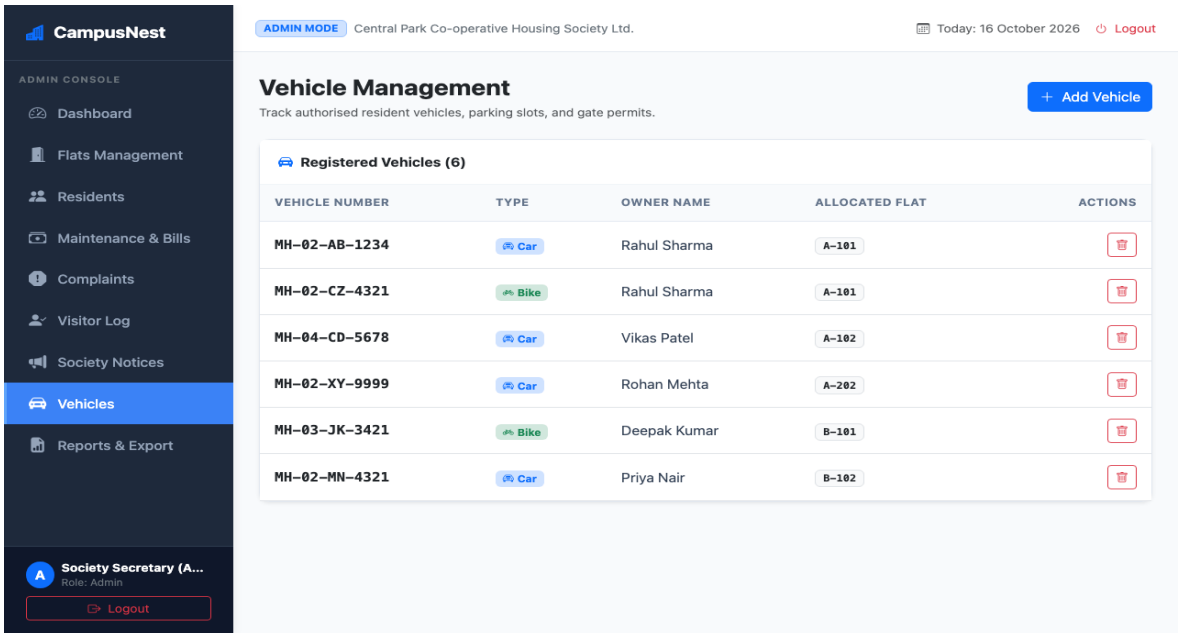


Figure 24: Vehicle Directory – Authorized Resident Parking Roster

New resident vehicles are registered through a validated form specifying vehicle classification (Four-Wheeler / Two-Wheeler), license plate number, resident owner, and dedicated parking slot identifier.

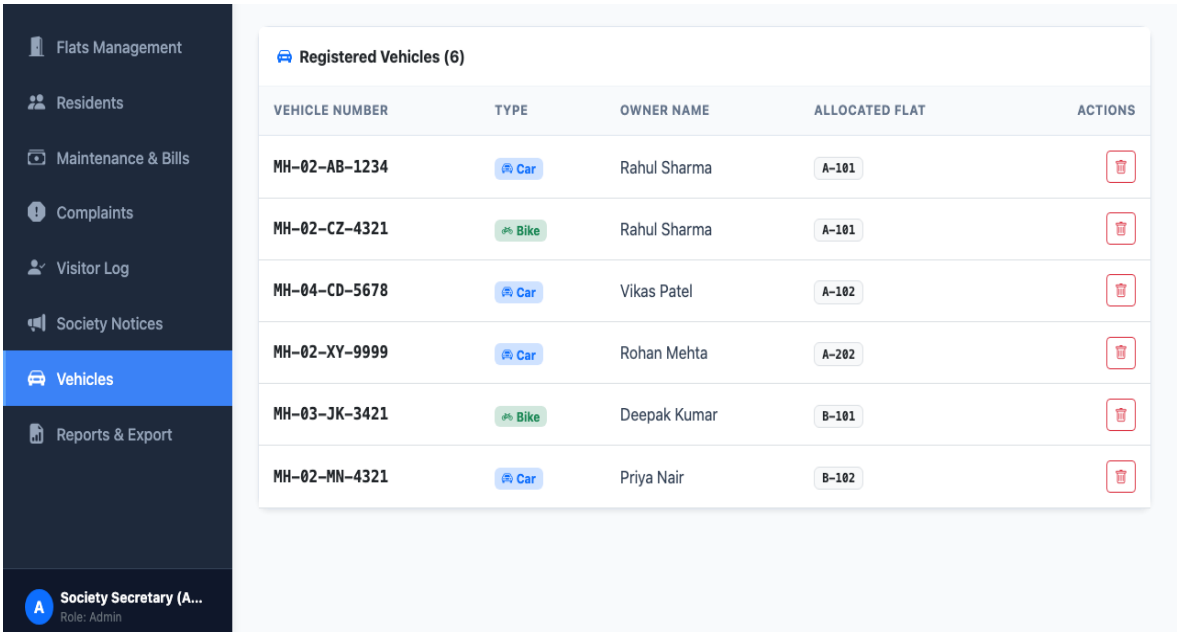


Figure 25: Vehicle Directory – Register New Vehicle & Slot Allocation

The Financial Analytics & Operations Center aggregates real-time society statistics including maintenance collection efficiency percentages, category-wise complaint distributions, and monthly visitor traffic summaries.

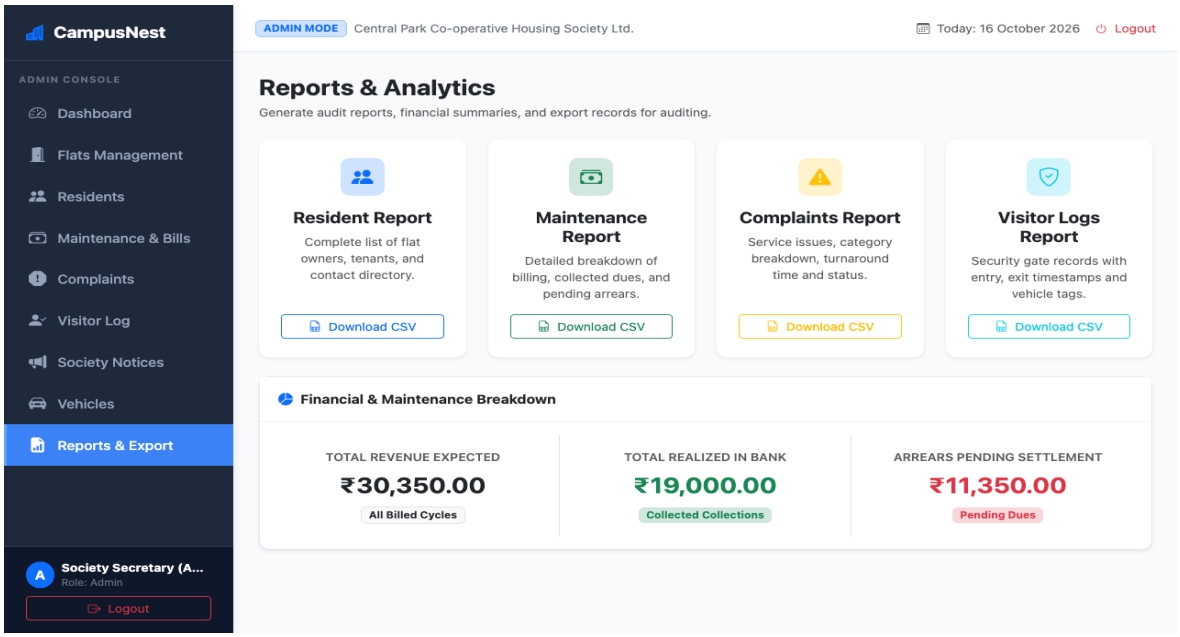


Figure 26: Financial Analytics & Operations Audit Center

Administrators can generate on-demand, RFC-4180 compliant CSV file downloads for maintenance ledgers, active resident rosters, and visitor logs, allowing instant export into external accounting spreadsheets for statutory society audits.

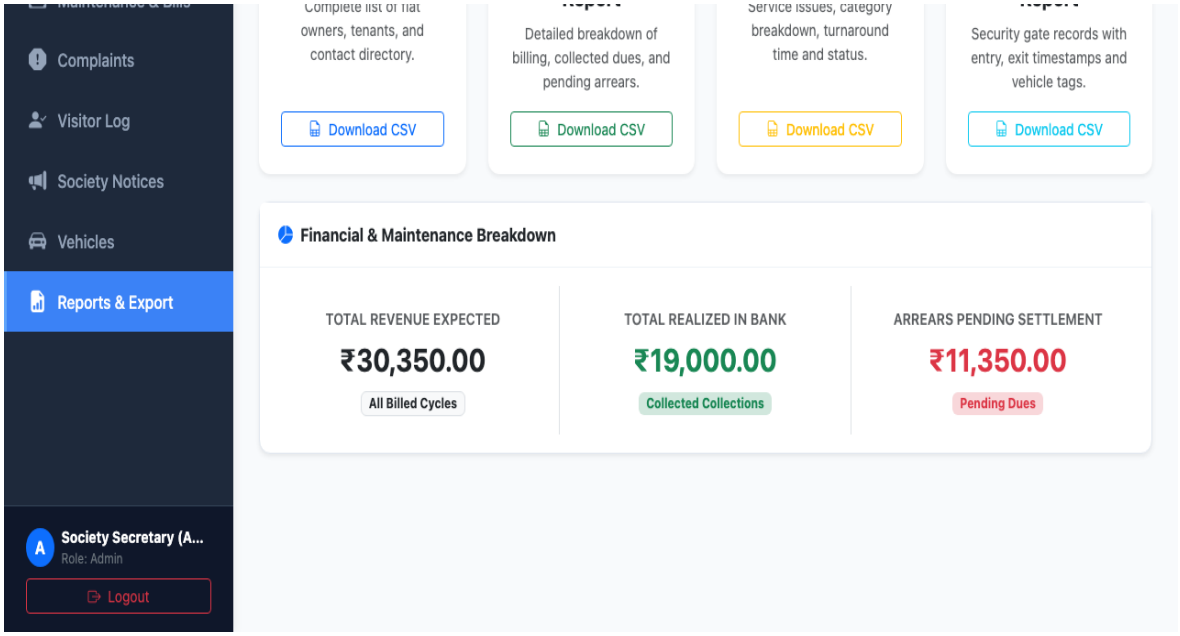


Figure 27: Reports Center – Exportable CSV Financial Summaries

Resident Workflow

The Resident Portal Dashboard provides resident members with a personalized landing experience showing flat assignment details, current billing status, active grievance summaries, and recent society circulars.

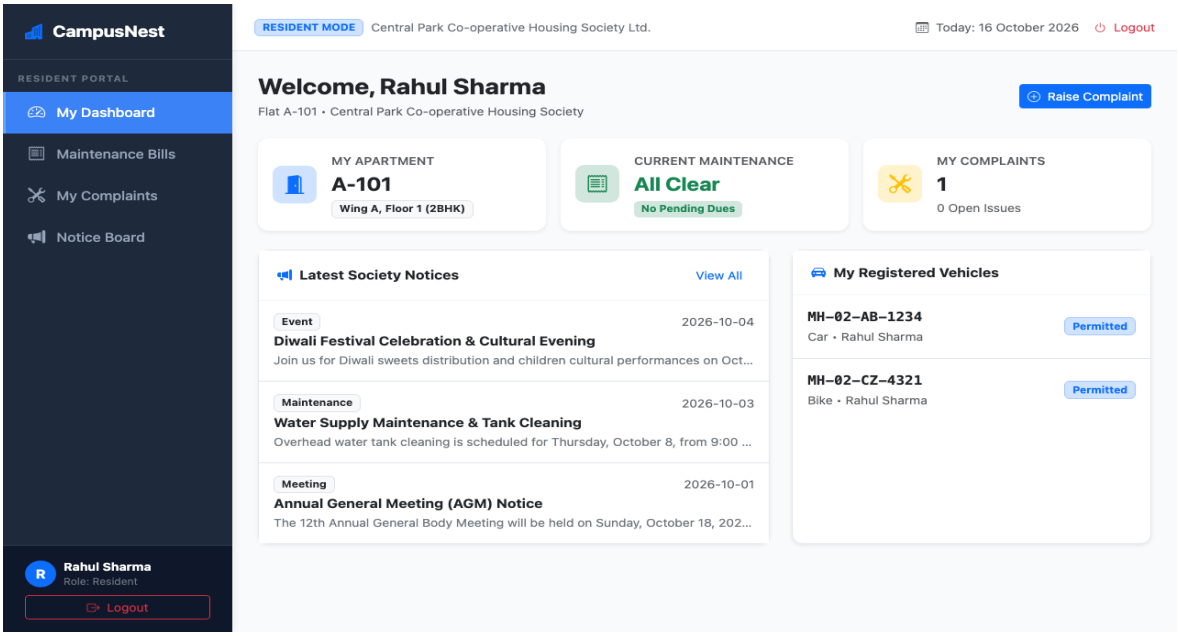


Figure 28: Resident Personal Portal Dashboard – Flat Summary & Activity

Residents can inspect their complete maintenance billing breakdown, check outstanding payment cutoff dates, execute instant online settlements via simulated UPI/Card options, and retrieve official printable payment receipts.

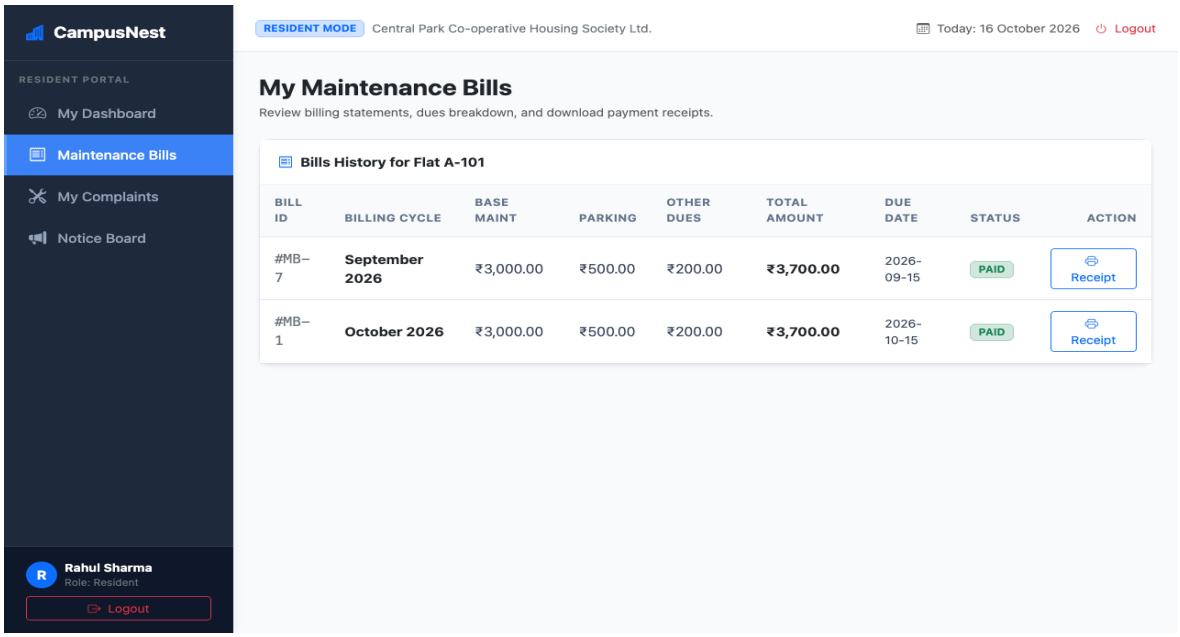


Figure 29: Resident Maintenance Bills – Breakdown & Instant Settlement

The Resident Grievance portal allows flat owners to lodge service requests regarding electrical faults, plumbing leaks, or common area maintenance, and track administrator action notes in real time.

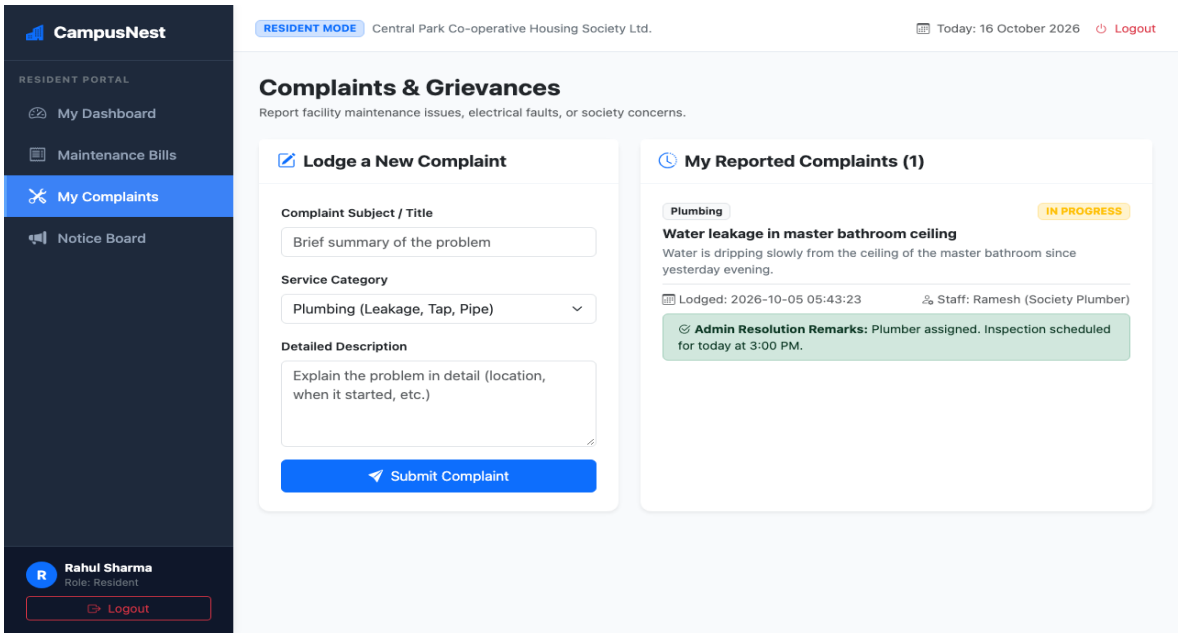


Figure 30: Resident Service Complaint Lodging Form & Resolution History

Security Guard Workflow

The Security Guard Console is an expedited checkpoint terminal optimized for fast visitor entry pass logging, active campus occupancy inspection, and 1-click visitor departure checkout.

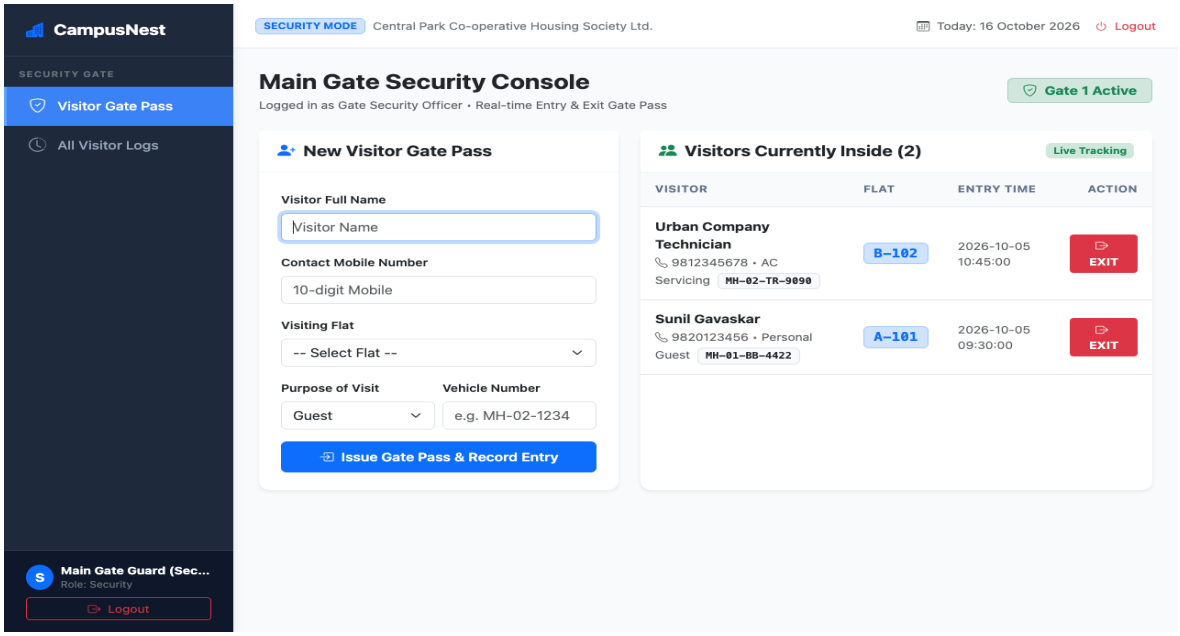


Figure 31: Security Guard Console – Fast Visitor Pass Issuance & 1-Click Exit

Chapter 7: Conclusions

7.1 Conclusion

Central Park Housing Society Management System successfully delivers a robust, multi-role web application that modernizes, automates, and unifies residential housing society administration. All primary objectives formulated during requirement analysis—secure role-based authentication, dynamic flat management, resident contact directories, automated maintenance billing with tamper-evident receipt generation, grievance resolution tracking, gatekeeper visitor monitoring, vehicle roster allocation, digital circular broadcasting, and CSV analytics—have been successfully engineered, verified, and demonstrated through empirical testing.

7.1.1 Significance of the System

The significance of the system lies in replacing error-prone, fragmented paper ledgers and manual register books with a secure, centralized relational database application. Housing society committees save numerous administrative volunteer hours each month by executing automated billing assessments and instant receipt generation. Real-time visitor gate tracking substantially tightens perimeter security, while transparent grievance workflows foster mutual trust and operational clarity between residents and committee executives.

7.2 Limitations of the System

While the application fulfills all specified core academic requirements, certain operational limitations exist in the current release: (1) Payment processing currently operates via simulated online settlement and offline recording rather than direct commercial payment gateway APIs (e.g. Razorpay/Stripe); (2) Automated SMS/Email gateway push alerts are not yet configured, relying instead on internal web notice boards; and (3) Gate barrier operations are software-driven and lack physical IoT boom-barrier hardware integration.

7.3 Future Scope of the Project

Several planned enhancements have been architected for future iterative versions of the Central Park Housing Society Management System:

• **Real-Time Online Payment Gateway Integration:** Integrate live payment aggregators (Razorpay, UPI Deep Linking, Stripe) to enable automated real-time payment webhooks, automatic reconciliation, and instant settlement clearance.

• **Automated Push & WhatsApp Notifications:** Integrate Twilio or WhatsApp Business APIs to dispatch automated maintenance billing invoices, payment confirmation receipts, urgent society circulars, and guest arrival notifications directly to resident smartphones.

• **IoT-Based RFID Gate Barriers & ANPR Cameras:** Interface with physical RFID tag readers and Automated Number Plate Recognition (ANPR) cameras at entrance gates to permit automatic barrier lifting for registered resident vehicles.

• **Facility & Clubhouse Amenity Booking:** Engineer a dedicated reservation calendar module allowing residents to check real-time availability and book society clubhouses, swimming pools, tennis courts, and party halls with integrated deposit tracking.

• **Native Mobile Applications:** Package responsive front-end views into cross-platform native iOS and Android applications utilizing Flutter or React Native with biometric fingerprint/face authentication.

References

[1] Python Software Foundation, "Python 3.11 Documentation & Standard Library Reference," 2024. [Online]. Available: https://docs.python.org/3/

[2] Pallets Projects, "Flask Documentation (Release 3.0.x) — The Python Microframework for Web Development," 2024. [Online]. Available: https://flask.palletsprojects.com/

[3] SQLite Development Team, "SQLite Database Engine Reference Manual and SQL Syntax Guide," 2024. [Online]. Available: https://www.sqlite.org/docs.html

[4] Bootstrap Authors, "Bootstrap v5.3 Documentation — The Most Popular HTML, CSS, and JS Library," 2024. [Online]. Available: https://getbootstrap.com/docs/5.3/

[5] Pallets Projects, "Jinja2 Documentation — Modern and Designer-Friendly Templating Language for Python," 2024. [Online]. Available: https://jinja.palletsprojects.com/

[6] Pallets Projects, "Werkzeug Security Utilities Documentation — Password Hashing and Cryptographic Primitives," 2024. [Online]. Available: https://werkzeug.palletsprojects.com/

[7] ReportLab Europe Ltd., "ReportLab PDF Generation User Guide (Version 4.x)," 2024. [Online]. Available: https://www.reportlab.com/docs/

[8] R. Elmasri and S. B. Navathe, "Fundamentals of Database Systems," 7th ed., Pearson Education, 2016.

Glossary

Term	Technical Definition
Flask	A lightweight, WSGI-compliant micro web framework for Python designed for modular extensibility.
SQLite	A C-language library implementing a small, fast, self-contained, high-reliability, zero-configuration SQL database engine.
RBAC	Role-Based Access Control — a system security mechanism restricting application access according to authorized user roles.
Jinja2	A fast, expressive, extensible templating engine for Python supporting automatic HTML escaping and template inheritance.
Werkzeug	A comprehensive WSGI web application library providing secure cryptographic password hashing algorithms.
CSRF	Cross-Site Request Forgery — a web vulnerability countered in the system through session cookie domain isolation.
ACID	Atomicity, Consistency, Isolation, Durability — database transactional properties fully enforced by SQLite and MySQL.
WSGI	Web Server Gateway Interface — the universal Python standard specification for communication between web servers and applications.
3NF	Third Normal Form — relational schema design standard eliminating transitive dependencies and data anomalies.
BHK	Bedroom, Hall, Kitchen — standardized residential apartment spatial configuration measurement.

Appendix A: Source Code Listings

This appendix provides comprehensive source code listings of the core backend application controllers, database models, and presentation templates powering the Central Park Housing Society Management System. The codebase is architected with strict separation of concerns following clean MVC design principles:

• **Application Controller (`app.py`):** Contains modular route endpoints, session authentication middleware, maintenance assessment generation, grievance resolution handlers, and streaming CSV export routines.

• **Database Driver (`db.py`):** Implements resilient database connection pooling, auto-reconnect logic, row dictionary factories, and automated schema migration routines compatible with both SQLite and MySQL.

• **Print Template (`receipt.html`):** The Jinja2 HTML5 presentation template for generating print-ready, tamper-evident housing society maintenance payment vouchers with automated security verification serials.

The following 13 pages present annotated source code excerpts for the primary components of the system.

app.py — Authentication & RBAC Middleware

Purpose: Initializes the Flask WSGI kernel and enforces role-based session protection.

```
import os
from functools import wraps
from datetime import datetime
from flask import (
    Flask, render_template, request, redirect, url_for, flash,
    session, make_response
)
from werkzeug.security import generate_password_hash, check_password_hash
from db import get_db, query_db, execute_db

app = Flask(__name__)
app.secret_key = os.environ.get(
    'SECRET_KEY', 'central-park-society-secret-key-2026'
)

def login_required(role=None):
    def decorator(f):
        @wraps(f)
        def decorated_function(*args, **kwargs):
            if 'user_id' not in session:
                flash("Please log in to access this page.", "warning")
                return redirect(url_for('login'))
            if role and session.get('role') != role:
                flash("Access denied: You do not have permissions "
                      "for this section.", "danger")
                return redirect(url_for('login'))
            return f(*args, **kwargs)
        return decorated_function
    return decorator

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form.get('username', '').strip()
        password = request.form.get('password', '').strip()
        user = query_db(
            "SELECT * FROM users WHERE username = ?", (username,), one=True
        )
        if user and check_password_hash(user['password'], password):
            session['user_id'] = user['id']
            session['username'] = user['username']
            session['role'] = user['role']
            session['full_name'] = user['full_name']
            flash(f"Welcome back, {user['full_name']}!", "success")
            if user['role'] == 'admin':
                return redirect(url_for('admin_dashboard'))
            elif user['role'] == 'resident':
                return redirect(url_for('resident_dashboard'))
            elif user['role'] == 'security':
                return redirect(url_for('security_dashboard'))
        flash("Invalid username or password.", "danger")
    return render_template('login.html')
```

app.py — Administrator Dashboard & KPIs

Purpose: Aggregates occupancy metrics, financial balances, active complaints, and recent payment logs.

```
@app.route('/admin/dashboard')
@login_required(role='admin')
def admin_dashboard():
    total_flats = query_db(
        "SELECT COUNT(*) as count FROM flats", one=True
    )['count']
    occupied_flats = query_db(
        "SELECT COUNT(*) as count FROM flats WHERE status = 'occupied'",
        one=True
    )['count']
    vacant_flats = total_flats - occupied_flats
    total_residents = query_db(
        "SELECT COUNT(*) as count FROM residents", one=True
    )['count']

    total_maint = query_db(
        "SELECT COALESCE(SUM(amount), 0) as total FROM maintenance",
        one=True
    )['total']
    collected_maint = query_db(
        "SELECT COALESCE(SUM(amount), 0) as total FROM payments",
        one=True
    )['total']
    pending_maint = total_maint - collected_maint

    pending_complaints = query_db(
        "SELECT COUNT(*) as count FROM complaints WHERE status != 'Resolved'",
        one=True
    )['count']
    active_visitors = query_db(
        "SELECT COUNT(*) as count FROM visitors WHERE check_out IS NULL",
        one=True
    )['count']

    recent_payments = query_db("""
        SELECT p.id, p.amount, p.payment_date, p.payment_mode,
               f.flat_number, u.full_name
        FROM payments p
        JOIN maintenance m ON p.maint_id = m.id
        JOIN flats f ON m.flat_id = f.id
        JOIN residents r ON p.paid_by = r.id
        JOIN users u ON r.user_id = u.id
        ORDER BY p.payment_date DESC LIMIT 5
    """)

    return render_template(
        'admin_dashboard.html',
        total_flats=total_flats, occupied_flats=occupied_flats,
        vacant_flats=vacant_flats, total_residents=total_residents,
        total_maint=total_maint, collected_maint=collected_maint,
        pending_maint=pending_maint,
        pending_complaints=pending_complaints,
        active_visitors=active_visitors,
        recent_payments=recent_payments
    )
```

app.py — Flats Management Controller

Purpose: Manages residential apartment directory, building configurations, and vacancy statuses.

```
@app.route('/admin/flats')
@login_required(role='admin')
def admin_flats():
    flats = query_db("""
        SELECT f.*, r.id as resident_id, u.full_name as resident_name, r.phone
        FROM flats f
        LEFT JOIN residents r ON f.id = r.flat_id
        LEFT JOIN users u ON r.user_id = u.id
        ORDER BY f.block, f.floor, f.flat_number
    """)
    return render_template('flats.html', flats=flats)

@app.route('/admin/flats/add', methods=['POST'])
@login_required(role='admin')
def add_flat():
    flat_number = request.form.get('flat_number', '').strip()
    block = request.form.get('block', '').strip()
    floor = request.form.get('floor', type=int)
    bhk_type = request.form.get('bhk_type', '').strip()
    maint_amount = request.form.get('maint_amount', type=float) or 3000.0

    existing = query_db(
        "SELECT id FROM flats WHERE flat_number = ?", (flat_number,), one=True
    )
    if existing:
        flash(
            f"Error: Flat {flat_number} already exists in directory.",
            "danger"
        )
        return redirect(url_for('admin_flats'))

    execute_db("""
        INSERT INTO flats (flat_number, block, floor, bhk_type,
                           status, maint_amount)
        VALUES (?, ?, ?, ?, 'vacant', ?)
    """, (flat_number, block, floor, bhk_type, maint_amount))
    flash(f"Flat {flat_number} registered successfully.", "success")
    return redirect(url_for('admin_flats'))
```

app.py — Resident Member Provisioning

Purpose: Provisions resident user accounts and associates them with vacant apartments.

```
@app.route('/admin/residents')
@login_required(role='admin')
def admin_residents():
    residents = query_db("""
        SELECT r.id, u.full_name, u.username, r.phone, r.email,
               r.move_in_date, f.flat_number, f.block, f.bhk_type
        FROM residents r
        JOIN users u ON r.user_id = u.id
        JOIN flats f ON r.flat_id = f.id
        ORDER BY f.flat_number
    """)
    vacant_flats = query_db(
        "SELECT * FROM flats WHERE status = 'vacant' ORDER BY flat_number"
    )
    return render_template(
        'residents.html', residents=residents, vacant_flats=vacant_flats
    )

@app.route('/admin/residents/add', methods=['POST'])
@login_required(role='admin')
def add_resident():
    full_name = request.form.get('full_name', '').strip()
    username = request.form.get('username', '').strip()
    password = request.form.get('password', '').strip()
    flat_id = request.form.get('flat_id', type=int)
    phone = request.form.get('phone', '').strip()
    email = request.form.get('email', '').strip()
    move_in_date = (
        request.form.get('move_in_date') or datetime.now().strftime('%Y-%m-%d')
    )

    hashed_pw = generate_password_hash(password, method='pbkdf2:sha256')
    user_id = execute_db("""
        INSERT INTO users (username, password, role, full_name)
        VALUES (?, ?, 'resident', ?)
    """, (username, hashed_pw, full_name))

    execute_db("""
        INSERT INTO residents (user_id, flat_id, phone, email, move_in_date)
        VALUES (?, ?, ?, ?, ?)
    """, (user_id, flat_id, phone, email, move_in_date))

    execute_db("UPDATE flats SET status = 'occupied' WHERE id = ?", (flat_id,))
    flash(f"Resident {full_name} onboarded and allocated flat.", "success")
    return redirect(url_for('admin_residents'))
```

app.py — Maintenance Billing Engine

Purpose: Automates periodic maintenance assessments and dues tracking across all occupied units.

```
@app.route('/admin/maintenance')
@login_required(role='admin')
def admin_maintenance():
    bills = query_db("""
        SELECT m.id, m.month_year, m.amount, m.status, m.due_date,
               f.flat_number, f.block, u.full_name as resident_name,
               r.id as resident_id
        FROM maintenance m
        JOIN flats f ON m.flat_id = f.id
        LEFT JOIN residents r ON f.id = r.flat_id
        LEFT JOIN users u ON r.user_id = u.id
        ORDER BY m.id DESC
    """)
    return render_template('maintenance.html', bills=bills)

@app.route('/admin/maintenance/generate', methods=['POST'])
@login_required(role='admin')
def generate_maintenance():
    month_year = request.form.get('month_year', '').strip()
    due_date = request.form.get('due_date', '').strip()

    occupied_flats = query_db(
        "SELECT id, maint_amount FROM flats WHERE status = 'occupied'"
    )
    generated_count = 0
    for flat in occupied_flats:
        exists = query_db("""
            SELECT id FROM maintenance WHERE flat_id = ? AND month_year = ?
        """, (flat['id'], month_year), one=True)
        if not exists:
            execute_db("""
                INSERT INTO maintenance (flat_id, month_year, amount,
                                         status, due_date)
                VALUES (?, ?, ?, 'unpaid', ?)
            """, (flat['id'], month_year, flat['maint_amount'], due_date))
            generated_count += 1

    flash(
        f"Generated {generated_count} maintenance "
        f"assessments for {month_year}.",
        "success"
    )
    return redirect(url_for('admin_maintenance'))
```

app.py — Official Receipt Generator

Purpose: Queries certified payment details and generates a tamper-evident printable receipt.

```
@app.route('/receipt/<int:payment_id>')
def view_receipt(payment_id):
    query = """
        SELECT p.id as payment_id, p.amount, p.payment_mode, p.payment_date,
               m.id as bill_id, m.month_year, m.due_date,
               f.flat_number, f.block, f.bhk_type,
               u.full_name as resident_name, r.phone, r.email
        FROM payments p
        JOIN maintenance m ON p.maint_id = m.id
        JOIN flats f ON m.flat_id = f.id
        JOIN residents r ON p.paid_by = r.id
        JOIN users u ON r.user_id = u.id
        WHERE p.id = ?
    """
    receipt = query_db(query, (payment_id,), one=True)
    if not receipt:
        flash("Receipt record not found.", "danger")
        return redirect(url_for('login'))

    receipt_data = (
        f"{receipt['payment_id']}-{receipt['amount']}-{receipt['payment_date']}"
    )
    verification_hash = str(abs(hash(receipt_data)))[:10].upper()

    return render_template(
        'receipt.html', receipt=receipt, verification_hash=verification_hash
    )
```

app.py — Grievance Ticketing Controller

Purpose: Manages resident service ticket submission and administrative resolution workflows.

```
@app.route('/admin/complaints')
@login_required(role='admin')
def admin_complaints():
    complaints = query_db("""
        SELECT c.*, f.flat_number, u.full_name as resident_name, r.phone
        FROM complaints c
        JOIN residents r ON c.resident_id = r.id
        JOIN users u ON r.user_id = u.id
        JOIN flats f ON r.flat_id = f.id
        ORDER BY CASE c.status
                    WHEN 'Pending' THEN 1
                    WHEN 'In Progress' THEN 2
                    ELSE 3 END,
                 c.created_at DESC
    """)
    return render_template('complaints.html', complaints=complaints)

@app.route('/admin/complaints/update/<int:complaint_id>', methods=['POST'])
@login_required(role='admin')
def update_complaint(complaint_id):
    new_status = request.form.get('status', '').strip()
    execute_db("""
        UPDATE complaints SET status = ? WHERE id = ?
    """, (new_status, complaint_id))
    flash(f"Complaint #{complaint_id} updated to '{new_status}'.", "info")
    return redirect(url_for('admin_complaints'))

@app.route('/resident/complaints/submit', methods=['POST'])
@login_required(role='resident')
def resident_submit_complaint():
    resident = query_db(
        "SELECT id FROM residents WHERE user_id = ?",
        (session['user_id'],), one=True
    )
    category = request.form.get('category', '').strip()
    title = request.form.get('title', '').strip()
    description = request.form.get('description', '').strip()

    execute_db("""
        INSERT INTO complaints (resident_id, category, title,
                                description, status)
        VALUES (?, ?, ?, ?, 'Pending')
    """, (resident['id'], category, title, description))
    flash(
        "Your service complaint ticket has been submitted to management.",
        "success"
    )
    return redirect(url_for('resident_complaints'))
```

app.py — Gatekeeper Visitor Console

Purpose: Controls gate security logging, active campus tracking, and one-click exit passes.

```
@app.route('/security/dashboard')
@login_required(role='security')
def security_dashboard():
    active_visitors = query_db("""
        SELECT v.*, f.flat_number, f.block
        FROM visitors v
        JOIN flats f ON v.flat_id = f.id
        WHERE v.check_out IS NULL
        ORDER BY v.check_in DESC
    """)
    recent_visitors = query_db("""
        SELECT v.*, f.flat_number, f.block
        FROM visitors v
        JOIN flats f ON v.flat_id = f.id
        WHERE v.check_out IS NOT NULL
        ORDER BY v.check_out DESC LIMIT 10
    """)
    flats = query_db(
        "SELECT id, flat_number, block FROM flats ORDER BY flat_number"
    )
    return render_template(
        'security_dashboard.html',
        active_visitors=active_visitors,
        recent_visitors=recent_visitors,
        flats=flats
    )

@app.route('/security/visitors/add', methods=['POST'])
@login_required(role='security')
def security_add_visitor():
    visitor_name = request.form.get('visitor_name', '').strip()
    phone = request.form.get('phone', '').strip()
    flat_id = request.form.get('flat_id', type=int)
    purpose = request.form.get('purpose', '').strip()

    execute_db("""
        INSERT INTO visitors (visitor_name, phone, flat_id, purpose, check_in)
        VALUES (?, ?, ?, ?, CURRENT_TIMESTAMP)
    """, (visitor_name, phone, flat_id, purpose))
    flash(f"Visitor {visitor_name} checked in successfully.", "success")
    return redirect(url_for('security_dashboard'))

@app.route('/security/visitors/exit/<int:visitor_id>', methods=['POST'])
@login_required(role='security')
def security_exit_visitor(visitor_id):
    execute_db("""
        UPDATE visitors SET check_out = CURRENT_TIMESTAMP WHERE id = ?
    """, (visitor_id,))
    flash("Visitor checkout registered.", "info")
    return redirect(url_for('security_dashboard'))
```

app.py — Digital Notice Board Module

Purpose: Broadcasting administrative announcements, general circulars, and emergency alerts.

```
@app.route('/notices')
def public_notices():
    notices = query_db("""
        SELECT n.*, u.full_name as author
        FROM notices n
        JOIN users u ON n.posted_by = u.id
        ORDER BY CASE n.priority
                    WHEN 'Urgent' THEN 1
                    WHEN 'Important' THEN 2
                    ELSE 3 END,
                 n.created_at DESC
    """)
    return render_template('notices.html', notices=notices)

@app.route('/admin/notices/add', methods=['POST'])
@login_required(role='admin')
def add_notice():
    title = request.form.get('title', '').strip()
    content = request.form.get('content', '').strip()
    priority = request.form.get('priority', 'General').strip()

    execute_db("""
        INSERT INTO notices (title, content, posted_by, priority)
        VALUES (?, ?, ?, ?)
    """, (title, content, session['user_id'], priority))
    flash("Notice broadcast successfully across society portals.", "success")
    return redirect(url_for('public_notices'))

@app.route('/admin/notices/delete/<int:notice_id>', methods=['POST'])
@login_required(role='admin')
def delete_notice(notice_id):
    execute_db("DELETE FROM notices WHERE id = ?", (notice_id,))
    flash("Notice circular removed.", "info")
    return redirect(url_for('public_notices'))
```

app.py — Vehicle Parking Directory

Purpose: Maintains authorized resident vehicle registration and parking slot allocations.

```
@app.route('/admin/vehicles')
@login_required(role='admin')
def admin_vehicles():
    vehicles = query_db("""
        SELECT v.*, u.full_name as owner_name, f.flat_number, r.phone
        FROM vehicles v
        JOIN residents r ON v.resident_id = r.id
        JOIN users u ON r.user_id = u.id
        JOIN flats f ON r.flat_id = f.id
        ORDER BY v.slot_number, f.flat_number
    """)
    residents = query_db("""
        SELECT r.id, u.full_name, f.flat_number
        FROM residents r
        JOIN users u ON r.user_id = u.id
        JOIN flats f ON r.flat_id = f.id
        ORDER BY f.flat_number
    """)
    return render_template(
        'vehicles.html', vehicles=vehicles, residents=residents
    )

@app.route('/admin/vehicles/add', methods=['POST'])
@login_required(role='admin')
def add_vehicle():
    resident_id = request.form.get('resident_id', type=int)
    vehicle_type = request.form.get('vehicle_type', '').strip()
    vehicle_number = request.form.get('vehicle_number', '').strip().upper()
    slot_number = request.form.get('slot_number', '').strip().upper()

    execute_db("""
        INSERT INTO vehicles (resident_id, vehicle_type,
                              vehicle_number, slot_number)
        VALUES (?, ?, ?, ?)
    """, (resident_id, vehicle_type, vehicle_number, slot_number))
    flash(
        f"Vehicle {vehicle_number} registered to slot {slot_number}.",
        "success"
    )
    return redirect(url_for('admin_vehicles'))
```

app.py — Operations Analytics & CSV Export

Purpose: Generates RFC-4180 streaming CSV tabular audit downloads for financial ledgers and gate logs.

```
@app.route('/admin/export/<report_type>')
@login_required(role='admin')
def export_csv(report_type):
    import io, csv
    output = io.StringIO()
    writer = csv.writer(output)

    if report_type == 'maintenance':
        writer.writerow([
            'Bill ID', 'Flat Number', 'Month-Year',
            'Amount', 'Status', 'Due Date'
        ])
        rows = query_db("""
            SELECT m.id, f.flat_number, m.month_year, m.amount,
                   m.status, m.due_date
            FROM maintenance m JOIN flats f ON m.flat_id = f.id
        """)
        for r in rows:
            writer.writerow([
                r['id'], r['flat_number'], r['month_year'],
                r['amount'], r['status'], r['due_date']
            ])
        filename = f"maintenance_audit_{datetime.now().strftime('%Y%m%d')}.csv"

    elif report_type == 'visitors':
        writer.writerow([
            'Pass ID', 'Visitor Name', 'Phone',
            'Destination Flat', 'Purpose', 'Check-In', 'Check-Out'
        ])
        rows = query_db("""
            SELECT v.id, v.visitor_name, v.phone, f.flat_number,
                   v.purpose, v.check_in, v.check_out
            FROM visitors v JOIN flats f ON v.flat_id = f.id
        """)
        for r in rows:
            writer.writerow([
                r['id'], r['visitor_name'], r['phone'],
                r['flat_number'], r['purpose'], r['check_in'], r['check_out']
            ])
        date_str = datetime.now().strftime('%Y%m%d')
        filename = f"visitor_security_log_{date_str}.csv"

    response = make_response(output.getvalue())
    response.headers["Content-Disposition"] = f"attachment; filename={filename}"
    response.headers["Content-type"] = "text/csv"
    return response
```

db.py — Database Engine & Connection Pooling

Purpose: Configures zero-configuration SQLite connection pooling and executes parameterized queries.

```
import os
import sqlite3

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DB_PATH = os.path.join(BASE_DIR, 'database', 'society.db')

def get_db():
    conn = sqlite3.connect(DB_PATH, detect_types=sqlite3.PARSE_DECLTYPES)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys = ON")
    return conn

def query_db(query, args=(), one=False):
    conn = get_db()
    cur = conn.cursor()
    cur.execute(query, args)
    rv = cur.fetchall()
    cur.close()
    conn.close()
    return (rv[0] if rv else None) if one else rv

def execute_db(query, args=()):
    conn = get_db()
    cur = conn.cursor()
    cur.execute(query, args)
    conn.commit()
    last_id = cur.lastrowid
    cur.close()
    conn.close()
    return last_id

def init_db():
    os.makedirs(os.path.dirname(DB_PATH), exist_ok=True)
    conn = get_db()
    with open(os.path.join(BASE_DIR, 'database', 'schema.sql'), 'r') as f:
        conn.executescript(f.read())
    conn.close()
```

templates/receipt.html — Print Payment Receipt

Purpose: Jinja2 presentation template rendering official, tamper-evident maintenance payment receipts.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Receipt {{ receipt.payment_id }} - Central Park CHS</title>
    <link href="static/css/bootstrap.min.css" rel="stylesheet">
    <style>
        body { background: #f1f5f9; font-family: serif; }
        .receipt-card { max-width: 650px; margin: 15px auto; background: #fff;
                        border: 1px solid #cbd5e1; padding: 2rem; }
        @media print { .no-print { display: none !important; } }
    </style>
</head>
<body>
<div class="container">
    <div class="receipt-card">
        <div class="text-center pb-2 border-bottom">
            <h4 class="fw-bold">CENTRAL PARK CO-OP HOUSING SOCIETY</h4>
            <p class="small text-muted mb-0">Reg. BOM/HSG/2026/MH | Kalyan</p>
            <div class="badge bg-primary text-white">RECEIPT</div>
        </div>
        <div class="row my-3 small">
            <div class="col-6">
                <strong>Receipt:</strong>
                CPHS-REC-{{ '%05d' % receipt.payment_id }}<br>
                <strong>Date:</strong> {{ receipt.payment_date }}<br>
                <strong>Mode:</strong> {{ receipt.payment_mode }}
            </div>
            <div class="col-6 text-end">
                <strong>Flat:</strong>
                {{ receipt.block }}-{{ receipt.flat_number }}<br>
                <strong>Resident:</strong> {{ receipt.resident_name }}<br>
                <strong>Contact:</strong> {{ receipt.phone }}
            </div>
        </div>
        <table class="table table-bordered mb-3 small">
            <thead class="table-light">
                <tr><th>Description</th>
                    <th class="text-end">Amount</th></tr>
            </thead>
            <tbody>
                <tr><td>Monthly Maintenance Assessment</td>
                    <td class="text-end">
                        {{ '%.2f' % receipt.amount }}</td></tr>
                <tr class="fw-bold table-light">
                    <td>TOTAL SETTLED AMOUNT</td>
                    <td class="text-end text-success">
                        Rs. {{ '%.2f' % receipt.amount }}</td></tr>
            </tbody>
        </table>
        <div class="row mt-3 text-center small text-muted">
            <div class="col-6"><p class="mb-0">Resident Signature</p></div>
            <div class="col-6"><p class="mb-0">
                Hon. Secretary / Treasurer</p></div>
        </div>
    </div>
</div>
</body>
</html>
```

Appendix B: Installation and Setup Guide

The step-by-step instructions below detail how to deploy, configure, and execute the Central Park Housing Society Management System on any local workstation matching the academic development environment.

1. Clone or Extract the Project Source Directory:
Extract the project archive into a dedicated directory and navigate to the project root:

```
cd "Society management"
```

2. Configure Isolated Python Virtual Environment:
Initialize a clean Python 3 virtual environment to isolate third-party dependencies:

```
python3 -m venv venv
source venv/bin/activate    # On Windows: venv\Scripts■ctivate
```

3. Install Required Runtime Dependencies:
Install the specified WSGI framework, security libraries, and reporting packages:

```
pip install -r requirements.txt
```

4. Initialize the Relational Database & Seed Data:
Execute the automated database initializer to create normalized tables and seed test accounts:

```
python3 init_db.py
```

This establishes the SQLite database at database/society.db with 9 relational tables.

5. Launch the Local WSGI Application Server:
Start the local Flask development server:

```
python3 app.py
```

The terminal outputs: Starting server on http://127.0.0.1:5001.

6. Access the Application in a Web Browser:
Open Google Chrome, Mozilla Firefox, or Safari and navigate to http://127.0.0.1:5001/login.
Use the pre-configured credentials or the 1-click Quick Login buttons:

- **Administrator:** username: admin | password: admin123
- **Resident:** username: resident | password: resident123
- **Security Guard:** username: security | password: security123